

U-Boot 分析

Joe

February 6, 2026

目录

1 工具	1
1.1 交叉编译工具链	1
1.2 调试工具 GDB	1
1.2.1 GDB 用法	1
1.2.2 GDB 命令	1
2 设备树	2
2.1 DTS&DTSI	2
2.1.1 节点 Node	3
2.1.2 属性 Property	3
2.1.3 其他	7
2.2 flat device tree	7
2.2.1 fdt header	8
2.2.2 fill area	8
2.2.3 dt_struct	8
2.2.4 dt_strings	9
2.3 API	9
2.3.1 寻找节点	9
2.3.2 读取属性	9
2.3.3 内存映射	10
2.3.4 解析中断	10
2.3.5 获取与节点对应的 platform_device	10
3 Kconfig	10
4 内存布局	10
5 启动流程	12
5.1 重要结构	12
5.1.1 全局变量	12
5.1.2 全局变量标志	16
5.1.3 存储分配	17
5.2 vector.S	18
5.3 start.S	18
5.4 crt0.S	19
5.4.1 调用 board_init_f_alloc_reserve	19
5.4.2 调用 board_init_f_init_reserve	20
5.4.3 调用 debug_uart_init	21
5.4.4 调用 board_f	22
5.4.5 调用 relocate_code	22
5.4.6 调用 c_runtime_cpu_setup	22
5.4.7 标志性输出	22
5.4.8 调用 board_init_r	22
5.5 board_f.c	22
5.5.1 init_sequence_f	23

5.5.2	setup_mon_len	26
5.5.3	fdtdec_setup	26
5.5.4	initf_malloc	27
5.5.5	initf_bootstage	27
5.5.6	initf_dm	28
5.5.7	env_init	30
5.5.8	init_baud_rate	31
5.5.9	serial_init	31
5.5.10	dram_init	31
5.5.11	reserve_xxx	31
5.5.12	reloc_xxx	32
5.5.13	setup_reloc	32
5.6	board_r.c	32
5.6.1	初始化列表	32
5.6.2	initr_flash	36
5.6.3	initr_dm	36
5.6.4	main_loop	36
6	驱动模型(DM)	37
6.1	生效配置	37
6.2	数据结构	37
6.2.1	uclass	38
6.2.2	uclass_driver	38
6.2.3	uclass_id	40
6.2.4	udevice	43
6.2.5	driver	44
6.2.6	udevice_id	46
6.2.7	driver_info	46
6.2.8	ofnode	46
6.2.9	device_node	46
6.2.10	ofprop	47
6.2.11	property	47
6.2.12	oftree	47
6.3	重要函数	47
6.3.1	dm_scan_plat	49
6.3.2	dm_extended_scan	49
6.3.3	dm_scan_other	50
6.3.4	dm_scan_fdt_node	50
6.3.5	device_bind_common	50
6.3.6	dm_probe_devices	54
6.4	重要驱动	56
6.4.1	pinctrl	56
7	环境变量(Environment)	60
8	命令(CMD)	62
8.1	重要结构	62
8.2	命令定义	63
8.3	命令使用	64

8.4	注意事项	65
8.5	重要命令	65
8.5.1	bootmenu	65
8.5.2	bootz	65
8.5.3	cp	65
8.5.4	env	65
8.5.5	go	66
8.5.6	loadx/y/b	66
8.5.7	loop	66
8.5.8	md	66
8.5.9	mm	66
8.5.10	printenv	66
8.5.11	reset	66
8.5.12	run	67
8.5.13	setenv	67
8.5.14	fdt	67
8.5.15	dm	67
9	事件(Event)	67
9.1	事件注册	67
9.2	事件调用	68
10	Trace	70
11	Stage	70
11.1	用法	70
11.2	原理	71
12	编译	72
12.1	u_boot_list	72
12.1.1	u_boot_list_2 类型	72
12.1.2	u_boot_list_1 类型	74
12.1.3	u_boot_list_3 类型	74
12.2	uclass_driver	74
12.3	driver_info	78

表格目录

Listings

1 工具

1.1 交叉编译工具链

- arm-none-eabi-xxx

1.2 调试工具 GDB

1.2.1 GDB 用法

- 适配器: JLink(Ubuntu 版本)
- GDB Server: JLinkGDBServerCLExe。

```
JLinkGDBServerCLExe -device CORTEX-A9 -if JTAG
```

- GDB: arm-none-eabi-gdb

```
arm-none-eabi-gdb -i=mi u-boot # u-boot is a elf file
```

1.2.2 GDB 命令

- 连接 Server:
- 加载 u-boot: load, 此时 pc 被设置为 entry 地址, 可以进行调试运行, 注意 u-boot 是带调试信息的 elf, 而 u-boot.elf 不带调试信息没法进行调试;
- file: 加载镜像, 包括符号表; file u-boot;
- symbol-file:
 1. 删除加载的镜像: symbol-file;
 2. 加载镜像, 而且解析符号地址时统一使用 relocaddr 基准: add-symbol-file u-boot relocaddr, relocaddr 在 gd_t 的全局数据结构中;
- restore: 在 gdb 环境下, 该指令可以把文件写入某个地址, 如把 dtb 写入地址 0x200000, restore dtb_xxx binary 0x200000. 其中地址 0x200000 可以用 gdb 导入的符号表符号代替;
- print: 可以输出符号表符号的值, 如下, 其中 \$ 引用寄存器 r9 的值; p ((gd_t*)\$r9)->dm_root
 1. p \$regname: 打印某个寄存器的值;
- i registers: 打印输出 core 的寄存器值
 1. i all-registers: 打印的寄存器范围更广
 2. i registers x1: 打印出某个寄存器的值;
- set print pretty on: 格式化打印
- set remotelogfile remote.log: remote server 的日志记录到文件
- set breakpoint auto-hw on: 断点自动设置为硬件断点
- set debug remote 1: 设置调试日志级别

2 设备树

device tree 分 2 种。

- 文本型的 dt, 解析起来比较慢, 读写友好;
- 二进制型的 fdt, 解析传输较快, 读写不友好

2.1 DTS&DTSI

DTS 文件是一个树状结构

- 结构上由 node 和 properties 构成;
- 每个 node 都有 1 个 parent node(除了根 node);
- 每个 node 包含若干 properties, 由 key-value 对构成;

DTSI 文件类似 c 语言的头文件, 可以被其他 dts/dtsi 文件包含;

```
1 #address-cells = <1>;
2 #size-cells = <1>;
3
4 aliases {
5     ethernet0 = &fec1;
6     i2c = &i2c1;
7 };
8
9 i2c1: i2c@30a20000 {
10     #address-cells = <1>;
11     #size-cells = <0>;
12     compatible = "fsl,imx8mm-i2c", "fsl,imx21-i2c";
13     reg = <0x0 0x30a20000 0x0 0x10000>;
14     interrupts = <GIC_SPI 35 IRQ_TYPE_LEVEL_HIGH>;
15     clocks = <&clk IMX8MM_CLK_I2C1_ROOT>;
16     status = "disabled";//没有激活
17 };
18
19 //对label i2c1的引用
20 &i2c1 {
21     clock-frequency = <400000>;
22     pinctrl-names = "default";
23     pinctrl-0 = <&pinctrl_i2c1>;//对label pinctrl_i2c1的引用
24     status = "okay";//激活
25     pmic: bd71837@4b {//孩子node, 表示pmic挂载i2c总线上
26         reg = <0x4b>;//i2c总线设备地址
27         compatible = "rohm,bd71840", "rohm,bd71837";
28         //PMIC BD71837 PMIC_nINT GPIO1_I03
29         pinctrl-0 = <&pinctrl_pmic>;
30         gpio_intr = <&gpio1 3 GPIO_ACTIVE_LOW>;
31         gpo {
32             rohm,drv = <0x0C>;
```

```

33     };
34 };
35 };

```

dtb 文件可以通过 dtc 程序转换为 dtb 格式的二进制文件, 命令为 `make dtbs`。DTC 的源代码位于内核的 `scripts/dtc` 目录中, 在 Linux 内核使能了设备树的情况下, 编译内核的时候主机工具 DTC 会被编译出来, 对应于 `scripts/dtc/Makefile` 中“`hostprogs-y:=dtc`”这个 `hostprogs` 的编译目标。当然, DTC 也可以在 Ubuntu 中单独安装, 命令为 `sudo apt-get install device-tree-compiler`。

DTC 也可以“反汇编”.dtb 文件为 .dts 文件, 其指令格式如下。 `dtc -I dtb -O dts -o xxx.dts xxx.dtb`。

2.1.1 节点 Node

例如上面一个 node。

- 标签 label: 如 `i2c1`, 标签可以在其他地方用来引用该 node;
- 节点名称 node name: 如 `i2c@30a20000`, 节点名称必须唯一, 如果该 node 没有地址, 可以不包含地址部分, name 使用“`ofnode_get_name`”读取;

特殊节点

- `aliases`: 用于 u-boot 或 kernel 中使用 1 个简短的名字来引用完整的节点路径, 但是在 dts 文件内部不能用它来引用节点(使用的是 label 来引用);
- `chosen`:
- `clocks`:
- `firmware`:

2.1.2 属性 Property

property 是 key-value 对, 其中 key 是由 1 到 31 个字符构成, value 可以取值如下。

- 空值 empty value: 此时只有 key, 表示该 key 是一个 bool 属性(true/false);
- 字符串 string value: 如 `status`;
- 字符串列表 string list: 如 `compatible`, 每个 value 值由逗号“,”隔开;
- 属性组 cell property array: 如上面的 `reg`、`gpio_intr`, 由“<...>”构成;
- 二进制数据 binary data: 如 `address=[00 1A F1 01 04 15]`, 由“[...]”构成;

具体的属性值, 可以由 API 读取“`ofnode_get_property()`”。

```

1  /{
2      compatible = "acme,coyotes-revenge";
3      #address-cells = <1>;
4      #size-cells = <1>;
5      interrupt-parent = <&intc>;
6
7      cpus {
8          #address-cells = <1>;
9          #size-cells = <0>;
10         cpu@0 {

```



```
11         compatible = "arm,cortex-a9";
12         reg = <0>;
13     };
14     cpu@1 {
15         compatible = "arm,cortex-a9";
16         reg = <1>;
17     };
18 };
19
20 serial@101f0000 {
21     compatible = "arm,pl011";
22     reg = <0x101f0000 0x1000 >;
23     interrupts = <1 0>;//由intc中的#interrupt-cells决定个数
24 };
25
26 serial@101f2000 {
27     compatible = "arm,pl011";
28     reg = <0x101f2000 0x1000 >;
29     interrupts = <2 0>;
30 };
31
32 gpio@101f3000 {
33     compatible = "arm,pl061";
34     reg = <0x101f3000 0x1000
35         0x101f4000 0x0010>;
36     interrupts = <3 0>;
37 };
38
39 intc: interrupt-controller@10140000 {
40     compatible = "arm,pl190";
41     reg = <0x10140000 0x1000 >;
42     interrupt-controller;
43     #interrupt-cells = <2>;
44 };
45
46 spi@10115000 {
47     compatible = "arm,pl022";
48     reg = <0x10115000 0x1000 >;
49     interrupts = <4 0>;
50 };
51
52 external-bus {
53     #address-cells = <2>
54     #size-cells = <1>;
55     ranges = <0 0/*子设备地址*/ 0x10100000/*cpu地址空间的地址*/ 0x10000/*空间length*/ // C
56             1 0/*子设备地址*/ 0x10160000/*cpu地址空间的地址*/ 0x10000/*空间length*/ // C
57             2 0/*子设备地址*/ 0x30000000/*cpu地址空间的地址*/ 0x4000000/
*空间length*/>; // Chipselect 3, NOR Flash
```

```

58
59     ethernet@0,0 {
60         compatible = "smc,smc91c111";
61         reg = <0 0 0x1000>;//address为0 0, length为0x1000
62         interrupts = <5 2>, <6 3>;
63         interrupt-names = "eth-tx", "eth-rx";//用名字命名5,6号中断
64     };
65
66     i2c@1,0 {
67         compatible = "acme,a1234-i2c-bus";
68         #address-cells = <1>;
69         #size-cells = <0>;
70         reg = <1 0 0x1000>;//address为1 0, length为0x1000
71         interrupts = <6 2>;
72         rtc@58 {
73             compatible = "maxim,ds1338";
74             reg = <58>;
75             interrupts = <7 3>;
76         };
77     };
78
79     flash@2,0 {
80         compatible = "samsung,k8f1315ebm", "cfi-flash";
81         reg = <2 0 0x4000000>;//address为2 0, length为0x4000000
82     };
83 };
84 };

```

status

- disabled: 表示该 node 在 kernel 中没有激活;
- okay: 表示激活, 另外没有 status 属性也等效 status=okay;

在 API 中, 读取 status 的状态判断该 node 是否启用。

```

1 int fdtdec_get_is_enabled(const void *blob, int node)
2 {
3     const char * cell;
4     cell = fdt_getprop(blob, node, "status", NULL);
5     if (cell)
6         return strcmp(cell, "okay") == 0;
7     return 1;
8 }

```

compatible 用于将设备和驱动绑定起来, 该字段由函数。来查找。

注意即使有 compatible 属性, 在 initf_dm 阶段还需要有 2.1.2 比如 bootph-all 才会进行加载, 否则只能在 initr_dm 阶段加载。

通常对于总线类节点 (如 SPI, IIC 等), 如果其下挂载了其他设备, 需要在 bind 时 (也就是 compatible 属性必须要有) 指定的 uclass_drv 的 post_bind 钩子函数中对下一级的 device 进行扫描绑定。

#address-cells 决定了 节点 reg 属性中地址信息所占 的字 (32 位), 比如章节2.1的 dts 描述中 i2c1 的 #address-cells 为 1, 表示子节点 pmic 中 reg 的起始地址所占用的字长为 1 个 word;

#size-cells 决定了 节点 reg 属性中 度信息所占的字 (32 位), 比如章节2.1的 dts 描述中 i2c1 的 #size-cells 为 1, 表示子节点 pmic 中 reg 的地址长度所占用的字长为 0 个 word, 没有长度的值, 相当于设置了起始地址而没有设置地址长度;

reg reg 属性都是和地址有关的内容, 有两种: 起始地址和地址 度, reg 属性的格式 为: reg = <addr1 len1 addr2 len2...>, 比如章节 2.1 的 i2c1 自身的 reg 属性, reg 的这个属性值表示从 addr 到 addrN+lenN-1 的地址范围都属于这个节点。

- #address-cells=<1>, #size-cells=<0>, 表示 length 字段为空;
- #address-cells=<2>, #size-cells=<1>, 表示形如 reg=<0 00 0x1000>;

ranges external-bus 的 ranges 属性定义了经过 external-bus 桥后的地址范围如何映射到 CPU 内存区域。ranges 是地址转换表, 其中的每个项 是个 地址、 地址以及在 地址空间的 小的映射。映射表中的 地址、 地址分别采 地址空间的 #address-cells 和 地址空间的 #address-cells 小。对于本例而 , 地址空间的 #address-cells 为 2, 地址空间的 #address-cells 值为 1, 因此对于“0 0 0x10100000 0x10000”

- 前 2 个 cell: 为 external-bus 桥后 external-bus 上 选 0 偏移 0, (地址)
- 第 3 个 cell 表 external-bus 上 选 0 偏移 0 的地址空间被映射到 CPU 的本地总线的 0x10100000 位置, (地址);
- 第 4 个 cell 表 映射的 小为 0x10000。ranges 后 两个项 的含义可以类推。(度 length);

#interrupt-cells #interrupt-cells 与 #address-cells 和 #size-cells 相似, 它表明连接此中断控制器的设备 (interrupt-controller) 的中断属性的 cell 小。

interrupt-parent 设备节点通过它来指定它所依附的中断控制器的 phandle, 当节点没有指定 interrupt-parent 时, 则从 级节点继承。对于本例而 , 根节点指定了 interrupt-parent=<&intc>; 其对应于“intc:interrupt-controller@10140000”, 而根节点的 节点并未指定 interrupt-parent, 因此它们都继承了 intc, 即位于 0x10140000 的中断控制器中。

interrupts 到了中断的设备节点, 通过它指定中断号、触发 法等, 这个属性具体含有多少个 cell, 由它依附的中断控制器节点的 #interrupt-cells 属性决定。

interrupt-names 可以通过 interrupt-names 来命令设备中的多个中断号, 并且用的 platform_get_irq_byname 来获取对应的中断, 比如

```
1 int eth_irq_init(struct platform_device *pdev, struct eth_engine * peth)
2 {
3     peth->txirq = platform_get_irq_byname(pdev, "eth-tx");
4     peth->rxirq = platform_get_irq_byname(pdev, "eth-rx");
5 }
```

#gpio-cells 类似于 interrupt-cells, 在 gpio-controller 里面定义需要用到 gpio 的 cell 数量, 比如 wp-gpios=<&gpio1 1>, 表示使用 GPIO1, 且为高电平, 类似 interrupts 属性。

clock-names 类似 interrupt-names

预加载属性 以下属性可以提前 `initf_dm` 中进行加载绑定, 否则需要在 `initr_dm` 中进行绑定。

- `bootph-all`
- `bootph-some-ram`
- `bootph-pre-ram`
- `bootph-pre-sram`
- `"u-boot,dm-pre-reloc"`
- `"u-boot,dm-pre-proper"`
- `"u-boot,dm-spl"`
- `"u-boot,dm-tpl"`
- `"u-boot,dm-vpl"`

2.1.3 其他

- 删除 node: 在某个已经定义好的 node 上加上这个, 可以删除该 node

```
1 &gpio1 {
2     /delete-node/ sd1_vselect_gpio; //编译时不使用
3 }
```

- 删除 property:

```
1 &gpio_buff{
2     hog_DIR_WL_DEV_WAKE{
3         gpio-hog;
4         gpios = <9 0>;
5         /delete-property/ output-low; //编译时不使用
6         output-high;
7     };
8 };
```

- 引用其他文件: `/include/ "soc.dtsi";`

2.2 flat device tree

fdt 结构如表所示, 整体分为 `fdt header`, `fill_area`, `dt_struct` 和 `dt_strings` 四个区域。

	magic(0xd00dfeed)	
	totalsize	
fdt header	off_dt_struct	dt_struct 的偏移
	off_dt_strings	dt_strings 的偏移
	off_mem_rsvmap	fill_area 的偏移
	version	
	last_comp_version	
	boot_cpuid_phys	
	size_dt_strings	dt_strings 的 size
	size_dt_struct	dt_struct 的 size
fill_area	0	fill 0
	...	
dt_struct	...	
dt_strings	...	

2.2.1 fdt header

flat 型 device tree 有一个头部结构叫 fdt_header, 其结构如下。

```
1 struct fdt_header {
2     fdt32_t magic;           // magic word FDT_MAGIC
3     fdt32_t totalsize;      // total size of DT block
4     fdt32_t off_dt_struct;   // offset to structure
5     fdt32_t off_dt_strings;  // offset to strings
6     fdt32_t off_mem_rsvmap;  // offset to memory reserve map
7     fdt32_t version;        // format version
8     fdt32_t last_comp_version; // last compatible version
9     // version 2 fields below
10    fdt32_t boot_cpuid_phys;  // Which physical CPU id we're booting on
11    // version 3 fields below
12    fdt32_t size_dt_strings;  // size of the strings block
13    // version 17 fields below
14    fdt32_t size_dt_struct;   // size of the structure block
15 };
```

2.2.2 fill area

全部填充 0.

2.2.3 dt_struct

- struct fdt_node_header: 节点(node)信息使用 fdt_node_header 结构体描述

```
1 struct fdt_node_header{
2     fdt32_t tag;
3     char name[0];
4 }
```

- struct fdt_property: 节点属性(property)使用 fdt_property 结构体描述

```
1 struct fdt_property {
```

```
2     fdt32_t tag;
3     fdt32_t len;
4     fdt32_t nameoff;
5     char data[0];
6 }
```

tag 是标识 node 的起始结束等信息的标志位, name 指向 node 名称的首地址, 有如下 tag

- FDT_BEGIN_NODE: 标识结点的起始
- FDT_END_NODE: 标识结点的结束;
- FDT_PROP: 标识结点下面的属性起始符
- FDT_NOP
- FDT_END

2.2.4 dt_strings

2.3 API

2.3.1 寻找节点

根据兼容属性, 获得设备节点。遍历设备树中的设备节点, 看看哪个节点的类型、兼容属性与本函数的输入参数匹配, 在多数情况下, from、type 为 NULL, 则表遍历了所有节点

```
1 //node定义
2 struct device_node {
3     const char *name;
4     const char *type;
5     phandle phandle;
6     const char *full_name;
7
8     struct property *properties;
9     struct device_node *parent;
10    struct device_node *child;
11    struct device_node *sibling;
12 };
13
14 struct device_node *of_find_compatible_node(struct device_node *from,
15                                             const char *type,
16                                             const char * compatible);
17 const char * ofnode_get_name(ofnode node)
```

2.3.2 读取属性

读取设备节点 np 的属性名, 为 propname, 属性类型为 8、16、32、64 位整型数组。

```
1 int of_property_read_u8_array(const struct device_node *np,
2                               const char *propname,
3                               u8 *out_values,
4                               size_t sz);
5 int of_property_read_u16_array(...);
```

```
6 int of_property_read_u32_array(...);
7 int of_property_read_u64(...);
```

2.3.3 内存映射

API 可以直接通过设备节点进入设备内存区间的 `ioremap()`, `index` 是内存段的索引。若设备节点的 `reg` 属性有多段, 可通过 `index` 标要 `ioremap()` 的是哪一段, 在只有 1 段的情况, `index` 为 0。采用设备树后, 一些设备驱动通过 `of_iomap()`, 而不再通过传统的 `ioremap()` 进行映射。

```
1 void __iomem *of_iomap(struct device_node * node, int index);
```

2.3.4 解析中断

通过设备树获得设备的中断号, 实际上是从 `.dts` 中的 `interrupts` 属性解析出中断号。若设备使用了多个中断, `index` 指定中断的索引号。

```
1 unsigned int irq_of_parse_and_map(struct device_node * dev, int index);
```

2.3.5 获取与节点对应的 `platform_device`

在可以拿到 `device_node` 的情况下, 如果想反向获取对应的 `platform_device`, 可使用 API。当然, 在已知 `platform_device` 的情况下, 想获取 `device_node` 则易如反掌。

```
1 struct platform_device *of_find_device_by_node(struct device_node *np);
```

3 Kconfig

编译完后, Kconfig 会生成文件 “`autoconf.h`” 头文件, 以下工具宏, 用于识别文件 “`./include/generated/autoconf.h`” 里面的宏定义。

- `CONFIG_VAL(macro)`: 得到宏配置 `macro` 的具体值;
- `IS_ENABLED(macro)`: 如果宏配置 `macro` 定义了返回 1, 否则 0;
- `IF_ENABLED_INT(option, int_option)`: 如果宏配置 `option` 定义了, 则返回宏配置 `int_option` 的值;
- `CONFIG_IS_ENABLED(macro)`: 和 `IS_ENABLED` 类似;
- `CONFIG_IF_ENABLED_INT(option, int_option)`: 和 `IF_ENABLED_INT` 类似;

4 内存布局

图1中是 U-Boot 的常见内存布局, 其实漏了一个部分, 就是在 Flash 中, 一般情况在紧跟 U-Boot 映像的后面, 还有一个存放环境变量的区域 (不过这个区域好像是可选可不选的), 一般都是在 Flash 中取一个 sector 来存放环境变量。

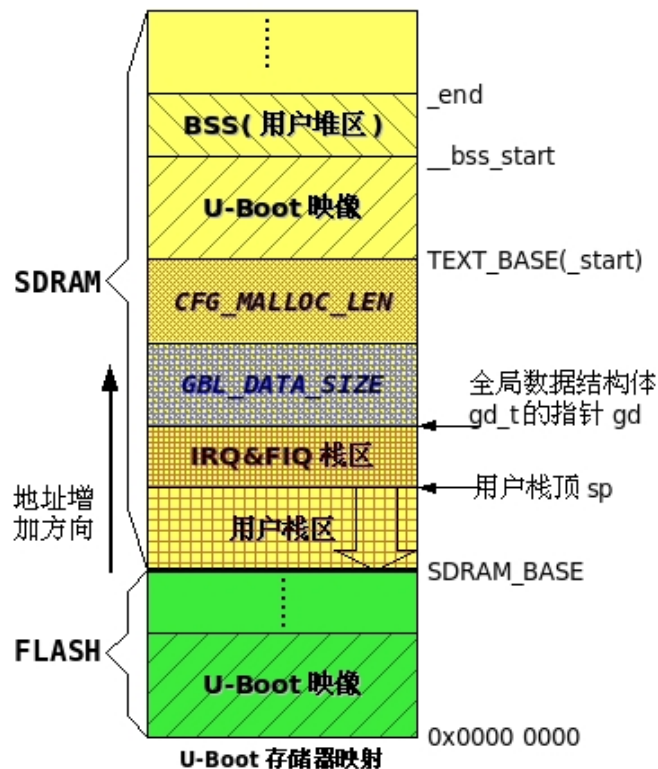


图 1: U-Boot 内存布局

- **U-Boot 映像:** U-Boot 烧写进 flash 的映像, 在 U-Boot 的术语中, 这部分的大小叫做 `monitor_size`. 所以在 U-Boot 中, 这个二进制文件就叫做 `monitor`. 这个 U-Boot 映像会被运送到 RAM 中, 从图中也可以看见 RAM 中有一块也是 U-Boot 映像。
- **环境变量区域:** 环境变量存放在 Flash 和 RAM 中各一份, 在 Flash 中一般存放在紧随 Monitor 也即 U-Boot 镜像的下一个 sector 中, 存储一些诸如 IP 地址等信息, 在程序被拷贝到 RAM 中时, 环境变量也同时被拷贝到 RAM 中。
- **CFG_MALLOC_LEN:** 这个区域是用来存放堆数据和环境变量的, 这个区域是紧接着 RAM 中的 U-Boot 镜像的, 从图中也可以看出, 在 U-Boot 的基地址往下开辟了这一段区域。环境变量在本来保存在 Flash 中, 在系统初始化的时候, flash 中的这些环境变量也同样被复制到 RAM 中, 在系统运行的时候, 可以修改 RAM 中的值来改变系统的环境变量, 但是掉电重启后, 还是用的 Flash 中的设定值, 当然也可以写 flash 来改变默认的环境变量。
- **GBL_DATA_SIZE:** 这个区域是紧接着 CFG_MALLOC_LEN 数据段的, 从图上也可以看出来, 这一段是用来存放一个 `gd_t` 数据结构的, 这个数据是一个 U-Boot 中用到的数据结构, 用来存放系统的一些信息
- 另外, 在 SDRAM_BASE 开始的地址上, 一般存放着二级跳转中断向量, 这些中断向量一般是用来为 uclinux 等操作系统用的。
- 其他的如栈的分布如图1所示。

5 启动流程

5.1 重要结构

5.1.1 全局变量

global_data 全局变量定义如下。

```
1 struct global_data {
2     struct bd_info * bd;//board information
3     unsigned long flags;//global data flags, See &enum gd_flags
4     unsigned int baudrate;//baud rate of the serial interface
5     unsigned long cpu_clk;//CPU clock rate in Hz
6     unsigned long bus_clk;//platform clock rate in Hz
7     unsigned long pci_clk;//PCI clock rate in Hz
8     unsigned long mem_clk;//memory clock rate in Hz
9     #if CONFIG_IS_ENABLED(VIDEO)
10    unsigned long fb_base;//base address of frame buffer memory
11    #endif
12    #if defined(CONFIG_POST)
13        //is a bit mask defining which POST tests are recorded
14        unsigned long post_log_word;//@post_log_word: active POST tests
15        //POST results is a bit mask with the POST results. A bit with value 1
16        //indicates successful execution.
17        unsigned long post_log_res;
18        unsigned long post_init_f_time;//time in ms when post_init_f() started
19    #endif
20    #ifdef CONFIG_BOARD_TYPES
21        //If a U-Boot configuration supports multiple board types, the actual
22        //board type may be stored in this field.
23        unsigned long board_type;//board type
24    #endif
25        //value 1 indicates that serial_init() was called and a console is available.
26        //value 0 indicates that console input and output drivers shall not be called.
27        unsigned long have_console;//console is available
28    #if CONFIG_IS_ENABLED(PRE_CONSOLE_BUFFER)
29        //pre-console buffer index, indicates the current position of the
30        //buffer used to collect output before the console becomes
31        //available. When negative, the pre-console buffer is
32        //temporarily disabled (used when the pre-console buffer is
33        //being written out, to prevent adding its contents to itself).
34        long precon_buf_idx;
35    #endif
36        //address of environment structure,contains the address of the
37        //structure holding the environment variables.
38        unsigned long env_addr;
39        unsigned long env_valid;//environment is valid
40        //bit mask indicating environment locations, defines which bit
41        //relates to which location
```

```
42     unsigned long env_has_init;
43     int env_load_prio;//priority of the loaded environment
44     unsigned long ram_base;//base address of RAM used by U-Boot
45     phys_addr_t ram_top;//top address of RAM used by U-Boot
46     //@relocaddr: start address of U-Boot in RAM
47     //After relocation this field indicates the address to which U-Boot
48     //has been relocated. It can be displayed using the bdfinfo command.
49     //Its value is needed to display the source code when debugging with
50     //GDB using the 'add-symbol-file u-boot <relocaddr>' command.
51     unsigned long relocaddr;
52     phys_size_t ram_size;//RAM size in bytes
53     unsigned long mon_len;//monitor length in bytes
54     unsigned long irq_sp;//IRQ stack pointer
55     unsigned long start_addr_sp;//initial stack pointer address
56     unsigned long reloc_off;//relocation offset
57     struct global_data *new_gd;//pointer to relocated global data
58 #ifdef CONFIG_DM
59     struct udevice *dm_root;//root instance for Driver Model
60     struct udevice *dm_root_f;//pre-relocation root instance
61     //head of core tree when uclasses are not in read-only memory.
62     //When uclasses are in read-only memory, @uclass_root_s is not used and
63     //points to the root node generated by dtoc.
64     struct list_head uclass_root_s;
65     //pointer to head of core tree, if uclasses are in read-only memory and
66     //cannot be adjusted to use @uclass_root as a list head.
67     //When not in read-only memory, @uclass_root_s is used to hold the
68     //uclass root, and @uclass_root points to the address of
69     //@uclass_root_s.
70     struct list_head *uclass_root;
71 # if CONFIG_IS_ENABLED(OF_PLATDATA_DRIVER_RT)
72     struct driver_rt *dm_driver_rt;//Dynamic info about the driver
73 # endif
74 #if CONFIG_IS_ENABLED(OF_PLATDATA_RT)
75     struct udevice_rt *dm_udevice_rt;//Dynamic info about the udevice
76     //Base address of the priv/plat region used when udevices and
77     //uclasses are in read-only memory. This is NULL if not used
78     void *dm_priv_base;
79 # endif
80 #endif
81 #ifdef CONFIG_TIMER
82     struct udevice *timer;//timer instance for Driver Model
83 #endif
84     const void *fdt_blob;//U-Boot's own device tree, NULL if none
85     void *new_fdt;//relocated device tree
86     unsigned long fdt_size;//space reserved for relocated device space
87     enum fdt_source_t fdt_src;//Source of FDT
88 #if CONFIG_IS_ENABLED(OF_LIVE)
89     struct device_node *of_root;//root node of the live tree
```

```
90 #endif
91 #if CONFIG_IS_ENABLED(MULTI_DTB_FIT)
92     const void *multi_dtb_fit;//pointer to uncompressed multi-dtb FIT image
93 #endif
94     //The jump table contains pointers to exported functions. A pointer to
95     //the jump table is passed to standalone applications.
96     struct jt_funcs *jt;
97     char env_buf[32];//buffer for env_get() before reloc
98 #ifdef CONFIG_TRACE
99     //trace buffer:When tracing function in U-Boot this field points
100     // to the buffer recording the function calls.
101     void *trace_buff;
102 #endif
103 #if CONFIG_IS_ENABLED(SYS_I2C_LEGACY)
104     int cur_i2c_bus;//currently used I2C bus
105 #endif
106     unsigned int timebase_h;//high 32 bits of timer
107     unsigned int timebase_l;//low 32 bits of timer
108 #if CONFIG_VAL(SYS_MALLOC_F_LEN)
109     unsigned long malloc_base;//base address of early malloc()
110     unsigned long malloc_limit;//limit address of early malloc()
111     unsigned long malloc_ptr;//current address of early malloc()
112 #endif
113 #ifdef CONFIG_PCI
114     struct pci_controller *hose;//PCI hose for early use
115     phys_addr_t pci_ram_top;//top of region accessible to PCI
116 #endif
117 #ifdef CONFIG_PCI_BOOTDELAY
118     //delay time before scanning of PIC hose expired
119     //If CONFIG_PCI_BOOTDELAY=y, pci_hose_scan() waits for the number of
120     //milliseconds defined by environment variable pcidelay before
121     //scanning. Once this delay has expired the flag @pcidelay_done is set to 1.
122     int pcidelay_done;
123 #endif
124     struct udevice *cur_serial_dev;//current serial device
125     struct arch_global_data arch;//architecture-specific data
126 #ifdef CONFIG_CONSOLE_RECORD
127     //@console_out: output buffer for console recording
128     //This buffer is used to collect output during console recording.
129     struct membuff console_out;
130     //input buffer for console recording, If console recording is
131     // activated, this buffer can be used to emulate input.
132     struct membuff console_in;
133 #endif
134 #if CONFIG_IS_ENABLED(VIDEO)
135     ulong video_top;//top of video frame buffer area
136     ulong video_bottom; //bottom of video frame buffer area
137 #endif
```

```
138 #ifdef CONFIG_BOOTSTAGE
139     struct bootstage_data *bootstage; //boot stage information
140     struct bootstage_data *new_bootstage; //relocated boot stage information
141 #endif
142 #ifdef CONFIG_LOG
143     //@log_drop_count: number of dropped log messages
144     //This counter is incremented for each log message which can not
145     //be processed because logging is not yet available as signaled by
146     //flag %GD_FLG_LOG_READY in @flags.
147     int log_drop_count;
148     //@default_log_level: default logging level
149     //For logging devices without filters @default_log_level defines the
150     //logging level, cf. &enum log_level_t.
151     int default_log_level;
152     struct list_head log_head; //list of logging devices
153     //@log_fmt: bit mask for logging format
154     //The @log_fmt bit mask selects the fields to be shown in log messages.
155     //@enum log_fmt defines the bits of the bit mask.
156     int log_fmt;
157     //@processing_msg: a log message is being processed
158     //This flag is used to suppress the creation of additional messages
159     //while another message is being processed.
160     bool processing_msg;
161     //@logc_prev: logging category of previous message
162     //This value is used as logging category for continuation messages.
163     int logc_prev;
164     //@logl_prev: logging level of the previous message
165     //This value is used as logging level for continuation messages.
166     int logl_prev;
167     //@log_cont: Previous log line did not finished with \n
168     //This allows for chained log messages on the same line
169     bool log_cont;
170 #endif
171 #if CONFIG_IS_ENABLED(BLOBLIST)
172     struct bloblist_hdr *bloblist; //blob list information
173     struct bloblist_hdr *new_bloblist; // relocated blob list information
174 #endif
175 #if CONFIG_IS_ENABLED(HANDOFF)
176     struct spl_handoff *spl_handoff; // SPL hand-off information
177 #endif
178 #if defined(CONFIG_TRANSLATION_OFFSET)
179     fdt_addr_t translation_offset; //optional translation offset
180 #endif
181 #ifdef CONFIG_ACPI
182     struct acpi_ctx *acpi_ctx; //ACPI context pointer
183     ulong acpi_start; //Start address of ACPI tables
184 #endif
185 #if CONFIG_IS_ENABLED(GENERATE_SMBIOS_TABLE)
```

```
186     char * smbios_version;//Points to SMBIOS type 0 version
187 #endif
188 #if CONFIG_IS_ENABLED(EVENT)
189     struct event_state event_state;//Points to the current state of events
190 #endif
191 #ifdef CONFIG_CYCLIC
192     struct hlist_head cyclic_list;//list of registered cyclic functions
193 #endif
194     struct list_head dmtag_list;//List of DM tags
195 };
```

5.1.2 全局变量标志

对全局变量操作时,有一些重要标志。

```
1  enum gd_flags {
2      //@GD_FLG_RELOC: code was relocated to RAM
3      GD_FLG_RELOC = 0x00001,
4      //@GD_FLG_DEVINIT: devices have been initialized
5      GD_FLG_DEVINIT = 0x00002,
6      //@GD_FLG_SILENT: silent mode
7      GD_FLG_SILENT = 0x00004,
8      //@GD_FLG_POSTFAIL: critical POST test failed
9      GD_FLG_POSTFAIL = 0x00008,
10     //@GD_FLG_POSTSTOP: POST sequence aborted
11     GD_FLG_POSTSTOP = 0x00010,
12     //@GD_FLG_LOGINIT: log Buffer has been initialized
13     GD_FLG_LOGINIT = 0x00020,
14     //@GD_FLG_DISABLE_CONSOLE: disable console (in & out)
15     GD_FLG_DISABLE_CONSOLE = 0x00040,
16     //@GD_FLG_ENV_READY: environment imported into hash table
17     GD_FLG_ENV_READY = 0x00080,
18     //@GD_FLG_SERIAL_READY: pre-relocation serial console ready
19     GD_FLG_SERIAL_READY = 0x00100,
20     //@GD_FLG_FULL_MALLOC_INIT: full malloc() is ready
21     GD_FLG_FULL_MALLOC_INIT = 0x00200,
22     //@GD_FLG_SPL_INIT: spl_init() has been called
23     GD_FLG_SPL_INIT = 0x00400,
24     //@GD_FLG_SKIP_RELOC: don't relocate
25     GD_FLG_SKIP_RELOC = 0x00800,
26     //@GD_FLG_RECORD: record console
27     GD_FLG_RECORD = 0x01000,
28     //@GD_FLG_RECORD_OVF: record console overflow
29     GD_FLG_RECORD_OVF = 0x02000,
30     //@GD_FLG_ENV_DEFAULT: default variable flag
31     GD_FLG_ENV_DEFAULT = 0x04000,
32     //@GD_FLG_SPL_EARLY_INIT: early SPL initialization is done
33     GD_FLG_SPL_EARLY_INIT = 0x08000,
34     //@GD_FLG_LOG_READY: log system is ready for use
```

```

35     GD_FLG_LOG_READY = 0x10000,
36     //@GD_FLG_CYCLIC_RUNNING: cyclic_run is in progress
37     GD_FLG_CYCLIC_RUNNING = 0x20000,
38     //@GD_FLG_SKIP_LL_INIT: don't perform low-level initialization
39     GD_FLG_SKIP_LL_INIT = 0x40000,
40     //@GD_FLG_SMP_READY: SMP initialization is complete
41     GD_FLG_SMP_READY = 0x80000,
42     //@GD_FLG_FDT_CHANGED: Device tree change has been detected by tests
43     GD_FLG_FDT_CHANGED = 0x100000,
44     //@GD_FLG_OF_TAG_MIGRATE: Device tree has old u-boot, dm- tags
45     GD_FLG_OF_TAG_MIGRATE = 0x200000,
46 };

```

5.1.3 存储分配

- sram

Item	Content	Size
1	malloc 分配区	0x1000Bytes
2	global_data 数据区	(216+15)&(~0xF) = 224Bytes
3	IRQ 栈区	
4	用户栈区	

- 由 Item1 到 Itemx 为 ram 地址减小方向;
- Item1 的顶部是 ram 的 top 地址, 长度 SYS_MALLOC_F_LEN, 可在 menuconfig 配置;
- Item2 的起始地址保存在 R9, 整个区域保存 u-boot 全局数据;

- dram(DDR)

Item	Content	Option	Size
1	video 占用空间	config	menuconfig 配置
2	trace 占用空间	config	menuconfig 配置
3	uboot relocate 空间	must	code 搬移的大小
4	malloc 分配区	must	和 sram 大小一致
5	struct bd_info 占用区	must	bd_info 大小
6	global_data 数据区	must	和 sram 大小一致
7	fdt 数据区	config	
8	bootstage 空间	config	
9	bloblist 空间	config	
10	arch 架构相关数据	config	
11	IRQ 栈区	must	16Bytes
12	用户栈区	must	
13	dtb		
14	kernel 空间		

- Item7: 如果为分离式的 fdt 则不需要, 如果是 embed 式, 具体放在符号表的“__dtb_dt_begin”代表的地址的后面;
- Item13: 如果采用分离式 dtb, 则在 u-boot 末尾存放 dtb, 具体放在符号表的“_end”代表的地址的后面;
- Item14: 为剩下的空间, 留给 kernel 使用。

5.2 vector.S

入口点在 vector.S (arch/arm/lib/vector.S), 该文件布局了中断向量, 也是 `_start` 标号的定义点。

```
1 _start:
2     ARM_VECTORS
3
4 .macro ARM_VECTORS
5 b reset    ;jump to start.S
6 ldr pc, _undefined_instruction
7 ldr pc, _software_interrupt
8 ldr pc, _prefetch_abort
9 ldr pc, _data_abort
10 ldr pc, _not_used
11 ldr pc, _irq
12 ldr pc, _fiq
13 .endm
```

5.3 start.S

经过 vector.S 的向量表定义后, 复位后将跳转到 start.S 的 reset 标号处, 进行多个关于 core 的初始化, 如 cache/mmu/hyper 等 core 相关的配置。

```
1     .globl reset
2 reset:
3     b save_boot_params
```

- 关闭 FIQ/IRQ, 进入 SVC 模式;

```
1 mrs r0, cpsr
2 and r1, r0, #0x1f @ mask mode bits
3 teq r1, #0x1a @ test for HYP mode
4 bicne r0, r0, #0x1f @ clear all mode bits
5 orrne r0, r0, #0x13 @ set SVC mode
6 orr r0, r0, #0xc0 @ disable FIQ and IRQ
7 msr cpsr,r0
```

- 设置 vector

```
1 @ Set V=0 in CP15 SCTLr register - for VBAR to point to vector
2 mrc p15, 0, r0, c1, c0, 0 @ Read CP15 SCTLr Register
3 bic r0, #CR_V @ V = 0
4 mcr p15, 0, r0, c1, c0, 0 @ Write CP15 SCTLr Register
5 @ Set vector address in CP15 VBAR register
6 ldr r0, =_start ;_start is entry point
7 mcr p15, 0, r0, c12, c0, 0 @Set VBAR
```

- 配置 CP15: bl cpu_init_cp15

– 禁止 cache

```

1 @Invalidate L1 I/D
2 mov r0, #0 @ set up for MCR
3 mcr p15, 0, r0, c8, c7, 0 @ invalidate TLBs
4 mcr p15, 0, r0, c7, c5, 0 @ invalidate icache
5 mcr p15, 0, r0, c7, c5, 6 @ invalidate BP array
6 dsb
7 isb

```

– 禁止 MMU

```

1 @ disable MMU stuff and caches
2 mrc p15, 0, r0, c1, c0, 0
3 bic r0, r0, #0x00002000 @ clear bits 13 (--V-)
4 bic r0, r0, #0x00000007 @ clear bits 2:0 (-CAM)
5 orr r0, r0, #0x00000002 @ set bit 1 (--A-) Align
6 orr r0, r0, #0x00000800 @ set bit 11 (Z---) BTB
7 bic r0, r0, #0x00001000 @ clear bit 12 (I) I-cache
8 mcr p15, 0, r0, c1, c0, 0

```

– 早期设置 SP

```

1 @Early stack for ERRATA that needs into call C code
2 ldr r0, =(SYS_INIT_SP_ADDR)
3 bic r0, r0, #7 @ 8-byte alignment for ABI compliance
4 mov sp, r0

```

- 做部分低级初始化 bl cpu_init_crit

```

1 ENTRY(lowlevel_init)
2     @ Enable the the VFP
3     mrc p15, 0, r1, c1, c0, 2
4     orr r1, r1, #(0x3 << 20)
5     orr r1, r1, #(0x3 << 20)
6     mcr p15, 0, r1, c1, c0, 2
7     isb
8     fmr r1, FPEXC
9     orr r1, r1, #(1<<30)
10    fmxr FPEXC, r1
11    @ Move back to caller
12    mov pc, lr
13 ENDPROC(lowlevel_init)

```

- 调用 crt0.S(arch/arm/lib/crt0.S)中的 _main: bl _main

5.4 crt0.S

5.4.1 调用 board_init_f_alloc_reserve

该函数用于预留 malloc 和全局变量 gd 的 ram 空间。

- top 参数传入的是原始栈顶地址。
- global_data 全局变量开辟在栈空间的高位段上。

- 返回的 top 修正后的地址重新写入 SP 寄存器。
- 并写入 R9 寄存器, 后续访问 global_data, 直接通过 R9 取得地址进行访问。

```
1  ulong board_init_f_alloc_reserve(ulong top)
2  {
3      //Reserve early malloc arena
4      #ifndef CFG_MALLOC_F_ADDR
5      #if CONFIG_VAL(SYS_MALLOC_F_LEN)
6          top -= CONFIG_VAL(SYS_MALLOC_F_LEN);
7      #endif
8      #endif
9      //LAST : reserve GD (rounded up to a multiple of 16 bytes)
10     top = rounddown(top-sizeof(struct global_data), 16);
11     return top;
12 }
```

在每个需要访问 global_data 全局变量的模块文件中, 都可以放入声明“DECLARE_GLOBAL_DATA_PTR”, 就可以通过 gd 指针访问 global_data。

```
#define DECLARE_GLOBAL_DATA_PTR    register volatile gd_t * gd asm ("r9")
```

5.4.2 调用 board_init_f_init_reserve

该函数用于初始化全局变量 gd(初始化为 0), 并重新设置 malloc 和 sp 地址。

- 先对 global_data 区域进行清零。
- 然后将 global_data 区域以上区域当作 malloc 分配区域
- 并将起始地址记载在 gd->malloc_base。

```
1  void board_init_f_init_reserve(ulong base)
2  {
3      struct global_data *gd_ptr;
4      //clear GD entirely and set it up.
5      //Use gd_ptr, as gd may not be properly set yet.
6      gd_ptr = (struct global_data *)base;
7      //zero the area
8      memset(gd_ptr, '\0', sizeof(*gd));
9      //set GD unless architecture did it already
10     if (CONFIG_IS_ENABLED(SYS_REPORT_STACK_F_USAGE))
11     {
12         board_init_f_init_stack_protection_addr(base);
13     }
14     //next alloc will be higher by one GD plus 16-byte alignment
15     base += roundup(sizeof(struct global_data), 16);
16     //record early malloc arena start.
17     //Use gd as it is now properly set for all architectures.
18     #if CONFIG_VAL(SYS_MALLOC_F_LEN)
19         //go down one 'early malloc arena'
20         gd->malloc_base = base;
```

```

21 #endif
22     if (CONFIG_IS_ENABLED(SYS_REPORT_STACK_F_USAGE))
23     {
24         board_init_f_init_stack_protection();
25     }
26 }

```

5.4.3 调用 debug_uart_init

该 uart 初始化可选配,用于早期 debug 使用 uart。

- board_debug_uart_init: 通常做模块时钟,引脚复用配置;
- _debug_uart_init: 进行 uart 初始化;

```

1 //Now define some functions - this should be inserted into the serial driver
2 #define DEBUG_UART_FUNCS \
3     static inline void _printch(int ch) \
4     { \
5         if (ch == '\n') \
6             _debug_uart_putc('\r'); \
7         _debug_uart_putc(ch); \
8     } \
9     void printch(int ch) \
10    { \
11        _printch(ch); \
12    } \
13    void printascii(const char * str) \
14    { \
15        while (* str) \
16            _printch(*str++); \
17    } \
18    static inline void printhex1(unsigned int digit) \
19    { \
20        digit &= 0xf; \
21        _debug_uart_putc(digit > 9 ? digit - 10 + 'a' : digit + '0'); \
22    } \
23    static inline void printhex(unsigned int value, int digits) \
24    { \
25        while (digits-- > 0) \
26            printhex1(value >> (4 * digits)); \
27    } \
28    void printhex2(unsigned int value) \
29    { \
30        printhex(value, 2); \
31    } \
32    void printhex4(unsigned int value) \
33    { \
34        printhex(value, 4); \
35    } \

```

```

36     void printhex8(unsigned int value)           \
37     {                                           \
38         printhex(value, 8);                     \
39     }                                           \
40     void printdec(unsigned int value)           \
41     {                                           \
42         if (value > 10) {                       \
43             printdec(value / 10);               \
44             value %= 10;                       \
45         } else if (value == 10) {               \
46             _debug_uart_putc('1');              \
47             value = 0;                         \
48         }                                       \
49         _debug_uart_putc('0' + value);          \
50     }                                           \
51     void debug_uart_init(void)                 \
52     {                                           \
53         board_debug_uart_init();                \
54         _debug_uart_init();                    \
55         _DEBUG_UART_ANNOUNCE                   \
56     }

```

5.4.4 调用 board_f

见5.5章节。

5.4.5 调用 relocate_code

做 relocate 操作

5.4.6 调用 c_runtime_cpu_setup

做 c 运行时准备

5.4.7 标志性输出

- coloured_LED_init
- red_led_on

5.4.8 调用 board_init_r

- 先做进入 board_init_r 的参数准备
 - 新的 gd_t 全局变量地址存入 r0;
 - relocate 地址存入 r1;
- 调用 board_init_r;

5.5 board_f.c

在_main中完成了上述配置后,就开始调用 board_init_f 做更多的板级模块初始化。实质就是顺序调用函数数组 init_sequence_f 里面的每个初始化序列函数。初化失败将被 hang 住。

```
1 void board_init_f(ulong boot_flags)
2 {
3     gd->flags = boot_flags;
4     gd->have_console = 0;
5     if (initcall_run_list(init_sequence_f))
6         hang();
7 }
```

5.5.1 init_sequence_f

init_sequence_f 初始化序列中包含若干初始化入口函数。

```
1 static const init_fnc_t init_sequence_f[] = {
2     setup_mon_len,
3     #ifdef CONFIG_OF_CONTROL
4     fdtdec_setup,
5     #endif
6     #ifdef CONFIG_TRACE_EARLY
7     trace_early_init,
8     #endif
9     initf_malloc,
10    log_init,
11    initf_bootstage,    // uses its own timer, so does not need DM
12    event_init,
13    #ifdef CONFIG_BLOBLIST
14    bloblist_init,
15    #endif
16    setup_spl_handoff,
17    #if defined(CONFIG_CONSOLE_RECORD_INIT_F)
18    console_record_init,
19    #endif
20    #if defined(CONFIG_HAVE_FSP)
21    arch_fsp_init,
22    #endif
23    arch_cpu_init,    // basic arch cpu dependent setup
24    mach_cpu_init,    // SoC/machine dependent CPU setup
25    initf_dm,
26    #if defined(CONFIG_BOARD_EARLY_INIT_F)
27    board_early_init_f,
28    #endif
29    #if defined(CONFIG_PPC) || defined(CONFIG_SYS_FSL_CLK) || defined(CONFIG_M68K)
30    // get CPU and bus clocks according to the environment variable
31    get_clocks,    // get CPU and bus clocks (etc.)
32    #endif
33    #if !defined(CONFIG_M68K) || (defined(CONFIG_M68K) && !defined(CONFIG_MCFTMR))
34    timer_init,    // initialize timer
35    #endif
36    #if defined(CONFIG_BOARD_POSTCLK_INIT)
37    board_postclk_init,
```

```
38 #endif
39     env_init,                // initialize environment
40     init_baud_rate, // initialize baudrate settings
41     serial_init,    // serial communications setup
42     console_init_f, // stage 1 init of console
43     display_options,        // say that we are here
44     display_text_info,      // show debugging info if required
45     checkcpu,
46 #if defined(CONFIG_SYSRESET)
47     print_resetinfo,
48 #endif
49 #if defined(CONFIG_DISPLAY_CPUINFO)
50     print_cpuinfo,        // display cpu info (and speed)
51 #endif
52 #if defined(CONFIG_DTB_RESELECT)
53     embedded_dtb_select,
54 #endif
55 #if defined(CONFIG_DISPLAY_BOARDINFO)
56     show_board_info,
57 #endif
58     INIT_FUNC_WATCHDOG_INIT
59     misc_init_f,
60     INIT_FUNC_WATCHDOG_RESET
61 #if CONFIG_IS_ENABLED(SYS_I2C_LEGACY)
62     init_func_i2c,
63 #endif
64 #if defined(CONFIG_VID) && !defined(CONFIG_SPL)
65     init_func_vid,
66 #endif
67     announce_dram_init,
68     dram_init, // configure available RAM banks
69 #ifndef CONFIG_POST
70     post_init_f,
71 #endif
72     INIT_FUNC_WATCHDOG_RESET
73 #if defined(CFG_SYS_DRAM_TEST)
74     testdram,
75 #endif // CFG_SYS_DRAM_TEST
76     INIT_FUNC_WATCHDOG_RESET
77 #ifndef CONFIG_POST
78     init_post,
79 #endif
80     INIT_FUNC_WATCHDOG_RESET
81     // Now that we have DRAM mapped and working, we can
82     // relocate the code and continue running from DRAM.
83     //
84     // Reserve memory at end of RAM for (top down in that order):
85     // - area that won't get touched by U-Boot and Linux (optional)
```

```
86     // - kernel log buffer
87     // - protected RAM
88     // - LCD framebuffer
89     // - monitor code
90     // - board info struct
91     setup_dest_addr,
92 #ifdef CONFIG_OF_BOARD_FIXUP
93     fix_fdt,
94 #endif
95 #ifdef CFG_PRAM
96     reserve_pram,
97 #endif
98     reserve_round_4k,
99     arch_reserve_mmu,
100    reserve_video,
101    reserve_trace,
102    reserve_uboot,
103    reserve_malloc,
104    reserve_board,
105    reserve_global_data,
106    reserve_fdt,
107    reserve_bootstage,
108    reserve_bloblist,
109    reserve_arch,
110    reserve_stacks,
111    dram_init_banksize,
112    show_dram_config,
113    INIT_FUNC_WATCHDOG_RESET
114    setup_bdinfo,
115    display_new_sp,
116    INIT_FUNC_WATCHDOG_RESET
117    reloc_fdt,
118    reloc_bootstage,
119    reloc_bloblist,
120    setup_reloc,
121 #if defined(CONFIG_X86) || defined(CONFIG_ARC)
122    copy_uboot_to_ram,
123    do_elf_reloc_fixups,
124 #endif
125    clear_bss,
126    // Deregister all cyclic functions before relocation, so that
127    // gd->cyclic_list does not contain any references to pre-relocation
128    // devices. Drivers will register their cyclic functions anew when the
129    // devices are probed again.
130    //
131    // This should happen as late as possible so that the window where a
132    // watchdog device is not serviced is as small as possible.
133    cyclic_unregister_all,
```

```
134 #if !defined(CONFIG_ARM) && !defined(CONFIG_SANDBOX) && \  
135     !CONFIG_IS_ENABLED(X86_64)  
136     jump_to_copy,  
137 #endif  
138     NULL,  
139 };
```

5.5.2 setup_mon_len

用于设置 global_data 中的 monitor length, 实质就是 __bss_end 段到 _start 的长度, 很奇怪, 不是整个 bin 的大小, 而只是一部分。

5.5.3 fdtdec_setup

flat device tree 描述符配置, 在 gd->fdt_blob 全局变量中记录 dtb 的起始地址(__dtb_dt_begin)。找到 fdt blob 地址后, 校验 fdt header 中的各种信息, 保证此 fdt 是合格有效的 device tree 数据, 具体见2章节。然后通过 fdtdec_board_setup 进行板级匹配(虽然此函数是空的)。

```
1 int fdtdec_setup(void)  
2 {  
3     int ret;  
4     // 根据fdt存放的类型, 找到fdt blob的存放地址  
5     //The devicetree is typically appended to U-Boot  
6     if (IS_ENABLED(CONFIG_OF_SEPARATE)) {  
7         gd->fdt_blob = fdt_find_separate();  
8         gd->fdt_src = FDTSRC_SEPARATE;  
9     } else { // embed dtb in ELF file for testing / development  
10        gd->fdt_blob = dtb_dt_embedded();  
11        gd->fdt_src = FDTSRC_EMBED;  
12    }  
13    // Allow the board to override the fdt address.  
14    if (IS_ENABLED(CONFIG_OF_BOARD)) {  
15        gd->fdt_blob = board_fdt_blob_setup(&ret);  
16        if (ret)  
17            return ret;  
18        gd->fdt_src = FDTSRC_BOARD;  
19    }  
20    // Allow the early environment to override the fdt address  
21    if (!IS_ENABLED(CONFIG_SPL_BUILD)) {  
22        ulong addr;  
23        //查找环境变量fdtcontroladdr的值  
24        addr = env_get_hex("fdtcontroladdr", 0);  
25        if (addr) {  
26            gd->fdt_blob = map_sysmem(addr, 0);  
27            gd->fdt_src = FDTSRC_ENV;  
28        }  
29    }  
30    if (CONFIG_IS_ENABLED(MULTI_DTB_FIT))  
31        setup_multi_dtb_fit();  
32    //根据找到的fdt blob地址, 进行各种参数校验, 如
```

```
33 //blob起始地址4字节对齐, fdt header头的各种参数校验 (magic key/version...)
34 ret = fdtdec_prepare_fdt(gd->fdt_blob);
35 if (!ret)
36     //校验通过后做各种板级配置, 目前为空操作
37     ret = fdtdec_board_setup(gd->fdt_blob);
38 //如果为MULTI_DTB_FIT配置, 则把blob地址存放到oftree_list数组中
39 oftree_reset();
40 return ret;
41 }
```

5.5.4 initf_malloc

设置 malloc 可用空间大小, 校验 malloc 起始地址。

```
1 int initf_malloc(void)
2 {
3     #if CONFIG_VAL(SYS_MALLOC_F_LEN)
4         assert(gd->malloc_base); // Set up by crt0.S
5         gd->malloc_limit = CONFIG_VAL(SYS_MALLOC_F_LEN);
6         gd->malloc_ptr = 0;
7     #endif
8     return 0;
9 }
```

5.5.5 initf_bootstage

初始化 u-boot 启动阶段, 记录各个阶段的时刻点, 便于后续做启动分析。

```
1 static int initf_bootstage(void)
2 {
3     bool from_spl = IS_ENABLED(CONFIG_SPL_BOOTSTAGE) &&
4         IS_ENABLED(CONFIG_BOOTSTAGE_STASH);
5     int ret;
6     //获取bootstage保存的变量(malloc分配)
7     ret = bootstage_init(!from_spl);
8     if (ret)
9         return ret;
10    if (from_spl) {
11        const void * stash = map_sysmem(CONFIG_BOOTSTAGE_STASH_ADDR,
12                                         CONFIG_BOOTSTAGE_STASH_SIZE);
13        ret = bootstage_unstash(stash, CONFIG_BOOTSTAGE_STASH_SIZE);
14        if (ret && ret != -ENOENT) {
15            debug("Failed to unstash bootstage: err=%d\n", ret);
16            return ret;
17        }
18    }
19    //记录下BOOTSTAGE_ID_START_UBOOT_F阶段的时刻点和ID名字
20    bootstage_mark_name(BOOTSTAGE_ID_START_UBOOT_F, "board_init_f");
21    return 0;
22 }
```


5.5.6 initf_dm

初始化 driver mode。具体见6章节,主要是函数 dm_init_and_scan

- dm_init: DM 初始化,实质就是建立 root 设备和 class_root(root 设备的操作类 class)。注意驱动分 device 的 driver 和 uclass 的 class_driver。
 - device_bind_by_name: 通过 driver_info->name 查找和 device 信息匹配的 driver,然后 calloc 创建对应的 udevice 和 uclass 并利用 driver 的 id 找到 uclass_driver,进行绑定,最后挂在 gd->dm_root 和 gd->uclass_root,并进一步把 driver 挂到创建的 udevice 上,把 class_driver 挂在 class 上。整个构造查找过程为:driver_info.name->driver.id->uclass_driver,而 device 和 uclass 都使用 calloc 在堆上生成,并挂接对应的驱动等元素,最终将 uclass_driver 汇集到 uclass 上,将 uclass 汇集到 udevice 上,将 driver 汇集到 udevice 上。
 1. lists_driver_lookup_name: 根据 name 查找(map 文件中的 driver 符号)
 2. device_bind_common: driver 和 udevice(指针挂接),class 和 class_driver(指针挂接),class 和 udevice 绑定(指针挂接)
 - (a) calloc 在堆里创建 device,dev,
 - (b) 初始化 device(INIT_LIST_HEAD);
 - (c) 挂接平台数据 dev_set_plat(dev,plat)
 - (d) 挂接驱动数据 dev->driver_data=driver_data
 - (e) 设置 device 名字 dev->name:实际就是查找 driver 的名字 name;
 - (f) 挂接 node,dev_set_ofnode(dev,node)
 - (g) 挂接 parent,dev->parent
 - (h) 挂接驱动 dev->driver(map 文件中找到的那个)
 - (i) 挂接 uclass,dev->uclass(driver 的 id 查找或在堆上新增的 class)
 - (j) calloc 申请 uclass_driver 的平台数据(per_device_plat_auto)
 - (k) 挂接到 dev 上,dev_set_uclass_plat(dev,ptr)
 - (l) 将 dev 挂到父设备(gd->dm_root 开始)的 child 列表中:list_add_tail(&dev->sibling_node, &parent->child_head);
 - (m) uclass_bind_device:uclass 绑定 device;
 - 1) 将 dev 所在的 uclass node 挂到 dev 的 uclass 列表中
list_add_tail(&dev->uclass_node, &uc->dev_head);
 - 2) 找到 dev 的 parent 的 uclass drive:dev->parent->uclass->uc_drv
 - 3) 执行 uc_drv->child_post_bind(dev) 回掉
 - (n) 回调 dev 自己驱动 bind 回掉 drv->bind://driver 与 device 绑定;
 - (o) 回调 parent 的 child_post_bind:parent->driver->child_post_bind(dev);
 - (p) 回调 uclass_driver 的 post_bind: uc->uc_drv->post_bind(dev);
 - (q) 最后设置上绑定标志:dev_or_flags(dev, DM_FLAG_BOUND);
 - device_probe: 对 root 设备执行 probe 操作;
- dm_scan_plat: 从平台设备中解析 udevice 和 uclass,实质就是从代码中静态定义的 driver_info 来绑定设备,和 root 设备绑定流程一致,从 map 文件中一堆定义的“U_BOOT_DRVINFO”, driver_info 来绑定 udevice
 - lists_bind_drivers
- dm_extended_scan_fdt: 从 dtb 中逐个解析 node(udevice) 和 driver 进行匹配挂接
 - dm_scan_fdt->dm_scan_fdt_node

1. fdtdec_get_is_enabled: status 状态判断 okay 和 disable, 没有 status 也是 okay
2. lists_bind_fdt: 实现 driver 和 udevice 的绑定, 只绑定具有 compatible 属性的 node, 总共 2 大循环嵌套, 第一层轮询 dts 里面的某个 node 的 compatible 里面的属性列表 compat1, 第二层轮训被使能的 driver (会在 map 文件中列出) 的 compatible 列表 compat2, 不停的用 compat1 去比较 compat2, 直到找到对应的 compat2, 就继续往下进行绑定。

```

1  int lists_bind_fdt(struct udevice * parent, ofnode node, struct udevice ** devp,
2                      struct driver *drv, bool pre_reloc_only)
3  {
4      struct driver *driver = ll_entry_start(struct driver, driver);
5      const int n_ents = ll_entry_count(struct driver, driver);
6      //...
7      compat_list = ofnode_get_property(node, "compatible", &compat_length);
8      //寻找compatible属性, 没有就返回
9      if (!compat_list) {
10         if (compat_length == -FDT_ERR_NOTFOUND) {
11             log_debug("Device '%s' has no compatible string\n", name);
12             return 0;
13         }
14         return compat_length;
15     }
16     //Walk through the compatible string list, attempting to match each
17     //compatible string in order such that we match in order of priority
18     //from the first string to the last.
19     //第1层循环, dts里面node的compatible列表
20     for (i = 0; i < compat_length; i += strlen(compat) + 1) {
21         compat = compat_list + i;
22         //第2层循环, 代码里面编译的所有compatible, 检查匹配
23         for (entry = driver; entry != driver + n_ents; entry++) {
24             ret = driver_check_compatible(entry->of_match, &id, compat);
25             if (!ret) break;
26         }
27         //检查pre_reloc属性
28         if (pre_reloc_only) {
29             if (!ofnode_pre_reloc(node) &&
30                 !(entry->flags & DM_FLAG_PRE_RELOC)) {
31                 log_debug("Skipping device pre-relocation\n");
32                 return 0;
33             }
34         }
35         //开始绑定device和driver
36         ret = device_bind_with_driver_data(parent, entry, name,
37                                             id ? id->data : 0,
38                                             node,
39                                             &dev);
40         break;
41     }
42     return result;
43 }

```

- (a) ofnode_get_name: 获取结点名称
 - (b) ofnode_get_property: 获取结点属性
 - (c) driver_check_compatible: 检查匹配情况
 - (d) device_bind_with_driver_data->device_bind_common: driver 和 udevice 绑定
- dm_scan_fdt_ofnode_path: 绑定特定的节点
1. "/chosen"
 2. "/clocks"
 3. "/firmware"
- dm_scan_other: 解析特殊 udevice 和 uclass
 - dm_probe_devices: 对扫描挂接后的所有设备及子设备, 进行 probe 调用生效。

```

1 list_for_each_entry(child, &dev->child_head, sibling_node)
2 {
3     dm_probe_devices(child, pre_reloc_only);
4 }

```

5.5.7 env_init

环境变量初始化操作, 通过遍历各个存储地的环境变量结构, 进行初始化, 如果没有, 则使用默认的环境变量 default_environment 列表来操作。

```

1 int env_init(void)
2 {
3     struct env_driver * drv;
4     int ret = -ENOENT;
5     int prio;
6     for (prio = 0; (drv = env_driver_lookup(ENVOP_INIT, prio)); prio++) {
7         if (!drv->init || !(ret = drv->init()))
8             env_set_initied(drv->location);
9         if (ret == -ENOENT)
10            env_set_initied(drv->location);
11         debug("%s: Environment %s init done (ret=%d)\n", __func__,
12             drv->name, ret);
13         if (gd->env_valid == ENV_INVALID)
14             ret = -ENOENT;
15     }
16     if (!prio)
17         return -ENODEV;
18     if (ret == -ENOENT) {
19         gd->env_addr = (ulong)&default_environment[0]; //默认环境变量
20         gd->env_valid = ENV_VALID;
21         return 0;
22     }
23     return ret;
24 }

```

5.5.8 init_baud_rate

获取环境变量里面的波特率, 如果没有, 就使用 menuconfig 里面设置的波特率。作为后级串口初始化时的波特率参数。

```
1 static int init_baud_rate(void)
2 {
3     gd->baudrate = env_get_ulong("baudrate", 10, CONFIG_BAUDRATE);
4     return 0;
5 }
```

5.5.9 serial_init

串口初始化。int serial_init(void) { #if CONFIG_IS_ENABLED(SERIAL_PRESENT) serial_find_console_or_panic();
gd->flags |= GD_FLG_SERIAL_READY; serial_setbrg(); #endif return 0; }

5.5.10 dram_init

- 第一步: 读取 dts 里面的 memory 节点
- 第二步: 本地 ddr 初始化

```
1 int dram_init(void)
2 {
3     //读取dts里面的memory节点
4     if (fdtdec_setup_mem_size_base() != 0)
5         return -EINVAL;
6     //本地ddr初始化
7     hwd_ddrc_init();
8     return 0;
9 }
```

5.5.11 reserve_xxx

- reserve_round_4k: 在 ddr(dram)上的 relocate 的起始地址需要 4K 对齐。
- reserve_video: 在 ddr(dram)上为 video 保留空间。
- reserve_trace: 在 ddr(dram)上为 trace 保留空间。
- reserve_uboot 在 ddr(dram)上为 relocate 保留空间。

```
1 static int reserve_uboot(void)
2 {
3     if (!(gd->flags & GD_FLG_SKIP_RELOC)) {
4         //reserve memory for U-Boot code, data & bss
5         //round down to next 4 kB limit
6         gd->relocaddr -= gd->mon_len;
7         gd->relocaddr &= ~(4096 - 1);
8     }
9     gd->start_addr_sp = gd->relocaddr;
10    return 0;
11 }
```

- reserve_malloc
- reserve_board
- reserve_global_data,
- reserve_fdt,
- reserve_bootstage,
- reserve_bloblist,
- reserve_arch,
- reserve_stacks,

5.5.12 reloc_xxx

- reloc_fdt: 如果非 embed 的 fdt,则需要做 relocate 到 dram(DDR) 中。
- reloc_bootstage: 在 relocate 前存的 bootstage 也要做 relocate。

5.5.13 setup_reloc

- 搬移 gd 数据到 relocate 区域的 gd 区。
- 其他计算;

5.6 board_r.c

5.6.1 初始化列表

类似 init_sequence_f, 逐个调用 init_sequence_r 里面的函数。注意里面有 WDT 的操作, 防止系统被复位。

```
1 static init_fnc_t init_sequence_r[] = {
2     initr_trace,
3     initr_reloc,
4     event_init,
5     #if defined(CONFIG_ARM) || defined(CONFIG_RISCV)
6     initr_caches,
7     //Note: For Freescale LS2 SoCs, new MMU table is created in DDR.
8     // A temporary mapping of IFC high region is since removed,
9     // so environmental variables in NOR flash is not available
10    // until board_init() is called below to remap IFC to high
11    // region.
12 #endif
13    initr_reloc_global_data,
14    #if IS_ENABLED(CONFIG_NEEDS_MANUAL_RELOC) && CONFIG_IS_ENABLED(EVENT)
15    event_manual_reloc,
16 #endif
17    #if defined(CONFIG_SYS_INIT_RAM_LOCK) && defined(CONFIG_E500)
18    initr_unlock_ram_in_cache,
19 #endif
20    initr_barrier,
```

```
21     initr_malloc,
22     log_init,
23     initr_bootstage, // Needs malloc() but has its own timer
24 #if defined(CONFIG_CONSOLE_RECORD)
25     console_record_init,
26 #endif
27 #ifdef CONFIG_SYS_NONCACHED_MEMORY
28     noncached_init,
29 #endif
30     initr_of_live,
31 #ifdef CONFIG_DM
32     initr_dm,
33 #endif
34 #ifdef CONFIG_ADDR_MAP
35     init_addr_map,
36 #endif
37 #if defined(CONFIG_ARM) || defined(CONFIG_RISCV) || defined(CONFIG_SANDBOX)
38     board_init, // Setup chipselects
39 #endif
40     //TODO: printing of the clock information of the board is now
41     //implemented as part of bdfinfo command. Currently only support for
42     //davinci SOC's is added. Remove this check once all the board
43     //implement this.
44 #ifdef CONFIG_CLOCKS
45     set_cpu_clk_info, // Setup clock information
46 #endif
47 #ifdef CONFIG_EFI_LOADER
48     efi_memory_init,
49 #endif
50     initr_binman,
51 #ifdef CONFIG_FSP_VERSION2
52     arch_fsp_init_r,
53 #endif
54     initr_dm_devices,
55     stdio_init_tables,
56     serial_initialize,
57     initr_announce,
58     dm_announce,
59 #if CONFIG_IS_ENABLED(WDT)
60     initr_watchdog,
61 #endif
62     INIT_FUNC_WATCHDOG_RESET
63 #if defined(CONFIG_NEEDS_MANUAL_RELOC) && defined(CONFIG_BLOCK_CACHE)
64     blkcache_init,
65 #endif
66 #ifdef CONFIG_NEEDS_MANUAL_RELOC
67     initr_manual_reloc_cmdtable,
68 #endif
```

```
69     arch_initr_trap,
70 #if defined(CONFIG_BOARD_EARLY_INIT_R)
71     board_early_init_r,
72 #endif
73     INIT_FUNC_WATCHDOG_RESET
74 #ifdef CONFIG_POST
75     post_output_backlog,
76 #endif
77     INIT_FUNC_WATCHDOG_RESET
78 #if defined(CONFIG_PCI_INIT_R) && defined(CONFIG_SYS_EARLY_PCI_INIT)
79     //Do early PCI configuration _before_ the flash gets initialised,
80     //because PCU resources are crucial for flash access on some boards.
81     pci_init,
82 #endif
83 #ifdef CONFIG_ARCH_EARLY_INIT_R
84     arch_early_init_r,
85 #endif
86     power_init_board,
87 #ifdef CONFIG_MTD_NOR_FLASH
88     initr_flash,
89 #endif
90     INIT_FUNC_WATCHDOG_RESET
91 #if defined(CONFIG_PPC) || defined(CONFIG_M68K) || defined(CONFIG_X86)
92     //initialize higher level parts of CPU like time base and timers
93     cpu_init_r,
94 #endif
95 #ifdef CONFIG_EFI_LOADER
96     efi_init_early,
97 #endif
98 #ifdef CONFIG_CMD_NAND
99     initr_nand,
100 #endif
101 #ifdef CONFIG_CMD_ONENAND
102     initr_onenand,
103 #endif
104 #ifdef CONFIG_MMC
105     initr_mmc,
106 #endif
107 #ifdef CONFIG_XEN
108     xen_init,
109 #endif
110 #ifdef CONFIG_PVBLOCK
111     initr_pvblock,
112 #endif
113     initr_env,
114 #ifdef CONFIG_SYS_MALLOC_BOOTPARAMS
115     initr_malloc_bootparams,
116 #endif
```

```
117     INIT_FUNC_WATCHDOG_RESET
118     cpu_secondary_init_r,
119 #if defined(CONFIG_ID_EEPROM)
120     mac_read_from_eeprom,
121 #endif
122     INIT_FUNC_WATCHDOG_RESET
123 #if defined(CONFIG_PCI_INIT_R) && !defined(CONFIG_SYS_EARLY_PCI_INIT)
124     //Do pci configuration
125     pci_init,
126 #endif
127     stdio_add_devices,
128     jumptable_init,
129 #ifdef CONFIG_API
130     api_init,
131 #endif
132     console_init_r, // fully init console as a device
133 #ifdef CONFIG_DISPLAY_BOARDINFO_LATE
134     console_announce_r,
135     show_board_info,
136 #endif
137 #ifdef CONFIG_ARCH_MISC_INIT
138     arch_misc_init, // miscellaneous arch-dependent init
139 #endif
140 #ifdef CONFIG_MISC_INIT_R
141     misc_init_r, // miscellaneous platform-dependent init
142 #endif
143     INIT_FUNC_WATCHDOG_RESET
144 #ifdef CONFIG_CMD_KGDB
145     kgdb_init,
146 #endif
147     interrupt_init,
148 #if defined(CONFIG_MICROBLAZE) || defined(CONFIG_M68K)
149     timer_init, // initialize timer
150 #endif
151 #if defined(CONFIG_LED_STATUS)
152     initr_status_led,
153 #endif
154     //PPC has a udelay(20) here dating from 2002. Why?
155 #ifdef CONFIG_BOARD_LATE_INIT
156     board_late_init,
157 #endif
158 #if defined(CONFIG_SCSI) && !defined(CONFIG_DM_SCSI)
159     INIT_FUNC_WATCHDOG_RESET
160     initr_scsi,
161 #endif
162 #ifdef CONFIG_BITBANGMII
163     bb_miiphy_init,
164 #endif
```



```
165 #ifdef CONFIG_PCI_ENDPOINT
166     pci_ep_init,
167 #endif
168 #ifdef CONFIG_CMD_NET
169     INIT_FUNC_WATCHDOG_RESET
170     initr_net,
171 #endif
172 #ifdef CONFIG_POST
173     initr_post,
174 #endif
175 #ifdef CONFIG_LAST_STAGE_INIT
176     INIT_FUNC_WATCHDOG_RESET
177     //Some parts can be only initialized if all others (like
178     //Interrupts) are up and running (i.e. the PC-style ISA
179     //keyboard).
180     last_stage_init,
181 #endif
182 #if defined(CFG_PRAM)
183     initr_mem,
184 #endif
185     run_main_loop,
186 };
```

5.6.2 initr_flash

- flash_init

5.6.3 initr_dm

5.6.4 main_loop

```
1 void main_loop(void)
2 {
3     const char *s;
4     bootstage_mark_name(BOOTSTAGE_ID_MAIN_LOOP, "main_loop");
5
6     if (IS_ENABLED(CONFIG_VERSION_VARIABLE))
7         env_set("ver", version_string); // set version variable
8
9     cli_init();
10    if (IS_ENABLED(CONFIG_USE_PREBOOT))
11        run_preboot_environment_command();
12    if (IS_ENABLED(CONFIG_UPDATE_TFTP))
13        update_tftp(OUL, NULL, NULL); // tftp
14    if (IS_ENABLED(CONFIG_EFI_CAPSULE_ON_DISK_EARLY)) {
15        //efi_init_early() already called
16        if (efi_init_obj_list() == EFI_SUCCESS)
17            efi_launch_capsules();
18    }
19    s = bootdelay_process(); // boot delay
```

```
20     if (cli_process_fdt(&s))
21         cli_secure_boot_cmd(s);
22
23     autoboot_command(s);
24     cli_loop();//command
25     panic("No CLI available");
26 }
```

6 驱动模型(DM)

u-boot 也引入了驱动模型(Driver Model: DM),这种驱动模型为驱动的定义和访问接口提供了统一的方法。提高驱动间的兼容性和访问的标准型,u-boot 驱动模型和 Linux kernel 的设备驱动模型类似。

6.1 生效配置

某个设备的驱动是否生效,需要若干步骤配置。

- 全局驱动模型开关,通过 menuconfig 设置“Device Drivers-> Generic Driver Options->Enable Driver Model”,最终配置在.config 文件中,CONFIG_DM=y。
- 特定驱动的开关,全局开关打开后,对于不同的驱动模块,可以使能或者禁能如 MMC,menuconfig 配置后,.config 文件中显示 CONFIG_DM_MMC=y
 - 在对应的源文件中,可以判断 `#if !CONFIG_IS_ENABLED(DM_MMC)`,来判断是否打开该驱动。
 - 在 Makefile 中,可以通过 `obj-$(CONFIG(SPL_)DM_MMC)+=mmc_uclass.o`,来判断是否将驱动模型加入编译过程。
- dts 文件中该设备的 node 需要有 compatible 属性;
- dts 文件中该设备的 node 需要有 status 属性且为 okay 状态,或者无 status 属性使用默认的生效状态;
- 有对应 node compatible 属性的 c 驱动代码,其中必须有 U_BOOT_DRIVER 定义的结构体,且 of_match 中包含这个 compatible 字符串;

6.2 数据结构

首先是 global_data 中 3 个管理驱动模型的变量。

- dm_root: 驱动模型的根设备,struct udevice 类型,管理 udevice 设备信息,通常需要根据设备信息找到驱动 driver,再挂接到这个根设备上(指针上),通过 API 进行绑定;
- dm_root_f: 重定向前的根设备,struct udevice 类型,管理 udevice 设备信息,进入 initr_dm 后会把 dm_root 备份到 dm_root_f;
- uclass_root: uclass 链表头,管理 uclass 驱动类,在 dm_init 中做如下动作

```
1 uc_drv = lists_uclass_lookup(id);//通过UCLASS_ROOT的ID找到uclass_driver
2 uc = calloc(1, sizeof(* uc));//申请uclass的存储空间
3 uc.uc_drv = uc_drv;//绑定uclass_driver到uclass
4 DM_UCLASS_ROOT_NON_CONST = uc//挂接uclass到gd_t全局变量
```

```

5 //uclass_bind_device中:
6 dev = calloc(1, sizeof(struct udevice));
7 dev->driver = drv;//查找到的_u_boot_list_2_driver_2_root_driver
8 dev->uclass = uc;//root设备挂接uclass
9 //将dev->uclass_node挂接到gd_t全局变量中uclass结构中
10 list_add_tail(&dev->uclass_node, &uc->dev_head);
11 //最后将申请的dev根设备挂到gd_t全部变量dm_root上
12 DM_ROOT_NON_CONST = dev

```

- uclass_root_s:

同一类的 udevice 由 uclass_id 归结为一类 uclass。uclass_driver 提供统一的操作接口来操作这类 udevice，也就是 uclass_driver 是操作 uclass 的驱动。而面对特定的 udevice 时，是由 device tree 描述定义，由 compatible 字符串结合识别，而 compatible 字符串又由 udevice_id 结构体来定义描述，并将 udevice 和 driver 绑定起来，driver 用于定义这个设备的具体操作。所有的 udevice 都挂接在 gd 的 dm_root 上，如果 udevice 还有子 udevice，则挂接在父 udevice 上。所有的 uclass_driver 都挂接在 gd 的 uclass_root 上。

6.2.1 uclass

uclass 定义管理某类型的驱动类别，如 GPIO、IIC，该类型下可以有若干个 udevice 的 driver，再比如 cortex-a9 内核的 global timer 和 wdt timer 就具有相同的操作类 uclass，通过 dev_head 进行链接，driver 操作接口，由 struct uclass_driver 统一定义（具体由 UCLASS_DRIVER 来定义）。若干个 uclass 构成了 u-boot 的驱动模型，uclass_driver 为每类 uclass 设备提供统一的操作接口，而 uclass_driver 中每个特定的 driver 由 U_BOOT_DRIVER 来定义，具体是某个厂商根据 uclass_driver 的操作接口要求而写的模块驱动程序。

```

1 struct uclass {
2     void *priv;//uclass的私有数据
3     struct uclass_driver * uc_drv;//uclass类的操作函数集合
4     struct list_head dev_head;//该uclass的所有设备
5     struct list_head sibling_node; //下一个uclass的节点
6 };

```

uclass 是 uboot 自动生成的，并且不是所有 uclass 都会生成，有对应 uclass_driver 并且有被 udevice 匹配到的 uclass 才会生成。所有生成的 uclass 都会被挂载 gd->uclass_root 链表上，直接遍历链表 gd->uclass_root 链表的 sibling_node 元素，并且根据 uclass_id 来获取到相应的 uclass，如果没有遍历到，就使用 uclass_add 根据 uclass_id 来添加对应的 uclass 到链表上。

```

// 从gd->uclass_root链表获取对应的uclass
int uclass_get(enum uclass_id key, struct uclass * * ucp);
// 根据id创建uclass，并在map中找到对应的uclass_drv挂接到uclass;
int uclass_add(enum uclass_id id, struct uclass * * ucp);

```

最终每类设备的 uclass 创建后都是新设备类靠前挂接在 gd->uclass_root 上，新设备类再挂接之前创建的旧的其他设备类。

6.2.2 uclass_driver

为类似的 device 提供统一的 driver 接口，主要通过 UCLASS_DRIVER 来定义。

```

1 struct uclass_driver {
2     const char * name; // 该uclass_driver的命令

```

```

3     enum uclass_id id; // 对应的uclass id
4     // 以下函数指针主要是调用时机的区别
5     int (*post_bind)(struct udevice * dev); // 在udevice被绑定到该uclass之后调用
6     int (*pre_unbind)(struct udevice * dev); // 在udevice被解绑出该uclass之前调用
7     int (*pre_probe)(struct udevice * dev); // 在该uclass的一个udevice进行probe之前
8     int (*post_probe)(struct udevice * dev); // 在该uclass的一个udevice进行probe之后
9     int (*pre_remove)(struct udevice * dev); // 在该uclass的一个udevice进行remove之前
10    int (*child_post_bind)(struct udevice * dev); // 在该uclass的一个udevice的一个子
11    int (*child_pre_probe)(struct udevice * dev); // 在该uclass的一个udevice的一个子
12    int (*init)(struct uclass * class); // 安装该uclass的时候调用
13    int (*destroy)(struct uclass * class); // 销毁该uclass的时候调用
14    int priv_auto_alloc_size; // 需要为对应的uclass分配多少私有数据
15    int per_device_auto_alloc_size; //
16    int per_device_platdata_auto_alloc_size; //
17    int per_child_auto_alloc_size; //
18    int per_child_platdata_auto_alloc_size; //
19    const void *ops; //操作集合
20    uint32_t flags; // 标识为
21 };

```

以 pinctrl 为例。

```

1  UCLASS_DRIVER(pinctrl) = {
2      .id = UCLASS_PINCTRL,
3      .post_bind = pinctrl_post_bind,
4      .flags = DM_UC_FLAG_SEQ_ALIAS,
5      .name = "pinctrl",
6  };
7  // Declare a new uclass_driver
8  #define UCLASS_DRIVER(_name) \
9      ll_entry_declare(struct uclass_driver, _name, uclass)
10 #define ll_entry_declare(_type, _name, _list)\
11     _type _u_boot_list_2_##_list##_2##_name __aligned(4)

```

最终转换出来如下,由结构体可得,其定义之后都被存放在了段._u_boot_list_2_uclass_2_pinctrl 中,

```

1 //同时存放在段._u_boot_list_2_uclass_2_pinctrl中, 也就是section段的内容
2 struct uclass_driver _u_boot_list_2_uclass_2_pinctrl = {
3     .id = UCLASS_PINCTRL,
4     .post_bind = pinctrl_post_bind,
5     .flags = DM_UC_FLAG_SEQ_ALIAS,
6     .name = "pinctrl",
7 }

```

从 u-boot.map 文件中可以看到,使用 uclass_1 和 uclass_3 来界定所有 driver 也就是 uclass_2 段

```

_u_boot_list_2_uclass_1 0x0000000013040c58 0x0 driver/build-in.o
_u_boot_list_2_uclass_2_blk 0x0000000013040c58 0x4c driver/build-in.o
                                0x0000000013040c58 _u_boot_list_2_uclass_2_blk
_u_boot_list_2_uclass_2_pinctrl 0x0000000013040c58 0x4c driver/build-in.o
                                0x0000000013040c58 _u_boot_list_2_uclass_2_pinctrl

```

```
.u_boot_list_2_uclass_2_gpio 0x0000000013040cxx 0x4c driver/build-in.o
                                0x0000000013040cxx _u_boot_list_2_uclass_2_gpio
...
.u_boot_list_2_uclass_3 0x0000000013040cxx 0x0 driver/build-in.o
```

6.2.3 uclass_id

uclass_id 用来区分不同类型的驱动。定义了若干种类型 ID

```
1  enum uclass_id {
2      // These are used internally by driver model
3      UCLASS_ROOT = 0,
4      UCLASS_DEMO,
5      UCLASS_TEST,
6      UCLASS_TEST_FDT,
7      UCLASS_TEST_FDT_MANUAL,
8      UCLASS_TEST_BUS,
9      UCLASS_TEST_PROBE,
10     UCLASS_TEST_DUMMY,
11     UCLASS_TEST_DEVRES,
12     UCLASS_TEST_ACPI,
13     UCLASS_SPI_EMUL, // sandbox SPI device emulator
14     UCLASS_I2C_EMUL, // sandbox I2C device emulator
15     UCLASS_I2C_EMUL_PARENT, // parent for I2C device emulators
16     UCLASS_PCI_EMUL, // sandbox PCI device emulator
17     UCLASS_PCI_EMUL_PARENT, // parent for PCI device emulators
18     UCLASS_USB_EMUL, // sandbox USB bus device emulator
19     UCLASS_AXI_EMUL, // sandbox AXI bus device emulator
20
21     // U-Boot uclasses start here - in alphabetical order
22     UCLASS_ACPI_PMC, // (x86) Power-management controller (PMC)
23     UCLASS_ADC, // Analog-to-digital converter
24     UCLASS_AHCI, // SATA disk controller
25     UCLASS_AUDIO_CODEC, // Audio codec with control and data path
26     UCLASS_AXI, // AXI bus
27     UCLASS_BLK, // Block device
28     UCLASS_BLKMAP, // Composable virtual block device
29     UCLASS_BOOTCOUNT, // Bootcount backing store
30     UCLASS_BOOTDEV, // Boot device for locating an OS to boot
31     UCLASS_BOOTMETH, // Bootmethod for booting an OS
32     UCLASS_BOOTSTD, // Standard boot driver
33     UCLASS_BUTTON, // Button
34     UCLASS_CACHE, // Cache controller
35     UCLASS_CLK, // Clock source, e.g. used by peripherals
36     UCLASS_CPU, // CPU, typically part of an SoC
37     UCLASS_CROS_EC, // Chrome OS EC
38     UCLASS_DISPLAY, // Display (e.g. DisplayPort, HDMI)
39     UCLASS_DMA, // Direct Memory Access
40     UCLASS_DSA, // Distributed (Ethernet) Switch Architecture
```

```
41      UCLASS_DSI_HOST, // Display Serial Interface host
42      UCLASS_ECDSA, // Elliptic curve cryptographic device
43      UCLASS_EFI_LOADER, // Devices created by UEFI applications
44      UCLASS_EFI_MEDIA, // Devices provided by UEFI firmware
45      UCLASS_ETH, // Ethernet device
46      UCLASS_ETH_PHY, // Ethernet PHY device
47      UCLASS_EXTCON, // External Connector Class
48      UCLASS_FIRMWARE, // Firmware
49      UCLASS_FPGA, // FPGA device
50      UCLASS_FUZZING_ENGINE, // Fuzzing engine
51      UCLASS_FS_FIRMWARE_LOADER, // Generic loader
52      UCLASS_FWU_MDATA, // FWU Metadata Access
53      UCLASS_GPIO, // Bank of general-purpose I/O pins
54      UCLASS_HASH, // Hash device
55      UCLASS_HWSPINLOCK, // Hardware semaphores
56      UCLASS_HOST, // Sandbox host device
57      UCLASS_I2C, // I2C bus
58      UCLASS_I2C_EEPROM, // I2C EEPROM device
59      UCLASS_I2C_GENERIC, // Generic I2C device
60      UCLASS_I2C_MUX, // I2C multiplexer
61      UCLASS_I2S, // I2S bus
62      UCLASS_IDE, // IDE device
63      UCLASS_IOMMU, // IOMMU
64      UCLASS_IRQ, // Interrupt controller
65      UCLASS_KEYBOARD, // Keyboard input device
66      UCLASS_LED, // Light-emitting diode (LED)
67      UCLASS_LPC, // x86 'low pin count' interface
68      UCLASS_MAILBOX, // Mailbox controller
69      UCLASS_MASS_STORAGE, // Mass storage device
70      UCLASS_MDIO, // MDIO bus
71      UCLASS_MDIO_MUX, // MDIO MUX/switch
72      UCLASS_MEMORY, // Memory Controller device
73      UCLASS_MISC, // Miscellaneous device
74      UCLASS_MMC, // SD / MMC card or chip
75      UCLASS_MOD_EXP, // RSA Mod Exp device
76      UCLASS_MTD, // Memory Technology Device (MTD) device
77      UCLASS_MUX, // Multiplexer device
78      UCLASS_NOP, // No-op devices
79      UCLASS_NORTHBRIDGE, // Intel Northbridge / SDRAM controller
80      UCLASS_NVME, // NVM Express device
81      UCLASS_NVMXIP, // NVM XIP devices
82      UCLASS_P2SB, // (x86) Primary-to-Sideband Bus
83      UCLASS_PANEL, // Display panel, such as an LCD
84      UCLASS_PANEL_BACKLIGHT, // Backlight controller for panel
85      UCLASS_PARTITION, // Logical disk partition device
86      UCLASS_PCH, // x86 platform controller hub
87      UCLASS_PCI, // PCI bus
88      UCLASS_PCI_EP, // PCI endpoint device
```

```
89      UCLASS_PCI_GENERIC, // Generic PCI bus device
90      UCLASS_PHY, // Physical Layer (PHY) device
91      UCLASS_PINCONFIG, // Pin configuration node device
92      UCLASS_PINCTRL, // Pinctrl (pin muxing/configuration) device
93      UCLASS_PMIC, // PMIC I/O device
94      UCLASS_POWER_DOMAIN, // (SoC) Power domains
95      UCLASS_PVBLOCK, // Xen virtual block device
96      UCLASS_PWM, // Pulse-width modulator
97      UCLASS_PWRSEQ, // Power sequence device
98      UCLASS_QFW, // QEMU firmware config device
99      UCLASS_RAM, // RAM controller
100     UCLASS_REBOOT_MODE, // Reboot mode
101     UCLASS_REGULATOR, // Regulator device
102     UCLASS_REMOTEPROC, // Remote Processor device
103     UCLASS_RESET, // Reset controller device
104     UCLASS_RNG, // Random Number Generator
105     UCLASS_RTC, // Real time clock device
106     UCLASS_SCMI_AGENT, // Interface with an SCMI server
107     UCLASS_SCSI, // SCSI device
108     UCLASS_SERIAL, // Serial UART
109     UCLASS_SIMPLE_BUS, // Bus with child devices
110     UCLASS_SMEM, // Shared memory interface
111     UCLASS_SOC, // SOC Device
112     UCLASS_SOUND, // Playing simple sounds
113     UCLASS_SPI, // SPI bus
114     UCLASS_SPI_FLASH, // SPI flash
115     UCLASS_SPI_GENERIC, // Generic SPI flash target
116     UCLASS_SPMI, // System Power Management Interface bus
117     UCLASS_SYSCON, // System configuration device
118     UCLASS_SYSINFO, // Device information from hardware
119     UCLASS_SYSRESET, // System reset device
120     UCLASS_TEE, // Trusted Execution Environment device
121     UCLASS_THERMAL, // Thermal sensor
122     UCLASS_TIMER, // Timer device
123     UCLASS_TPM, // Trusted Platform Module TIS interface
124     UCLASS_UFS, // Universal Flash Storage
125     UCLASS_USB, // USB bus
126     UCLASS_USB_DEV_GENERIC, // USB generic device
127     UCLASS_USB_HUB, // USB hub
128     UCLASS_USB_GADGET_GENERIC, // USB generic device
129     UCLASS_VIDEO, // Video or LCD device
130     UCLASS_VIDEO_BRIDGE, // Video bridge, e.g. DisplayPort to LVDS
131     UCLASS_VIDEO_CONSOLE, // Text console driver for video device
132     UCLASS_VIDEO_OSD, // On-screen display
133     UCLASS_VIRTIO, // VirtIO transport device
134     UCLASS_W1, // Dallas 1-Wire bus
135     UCLASS_W1_EEPROM, // one-wire EEPROMs
136     UCLASS_WDT, // Watchdog Timer driver
```

```
137
138         UCLASS_COUNT,
139         UCLASS_INVALID = -1,
140     };
```

6.2.4 udevice

udevice 为驱动模型最终操作的对象。代码中调用 U_BOOT_DEVICE 宏来定义设备资源, 实际为一个设备实例。将设备描述信息写在对应的 DTS 文件中, 然后编译成 DTB, 最终由 uboot 解析设备树后动态生成的。

```
1  struct udevice {
2      const struct driver *driver; //device对应的driver
3      const char *name; //driver的名称
4      void *plat_;
5      void *parent_plat_;
6      void *uclass_plat_;
7      ulong driver_data;
8      struct udevice *parent; //父设备
9      void *priv_; //私有数据指针
10     struct uclass *uclass; //驱动所属的uclass
11     void *uclass_priv_;
12     void *parent_priv_;
13     struct list_head uclass_node;
14     struct list_head child_head;
15     struct list_head sibling_node;
16     #if !CONFIG_IS_ENABLED(OF_PLATDATA_RT)
17         u32 flags_;
18     #endif
19     int seq_;
20     #if CONFIG_IS_ENABLED(OF_REAL)
21         ofnode node_; //设备树节点
22     #endif
23     #if CONFIG_IS_ENABLED(DEVRES)
24         struct list_head devres_head;
25     #endif
26     #if CONFIG_IS_ENABLED(DM_DMA)
27         ulong dma_offset;
28     #endif
29     #if CONFIG_IS_ENABLED(IOMMU)
30         struct udevice *iommu;
31     #endif
32 };
```

最终 udevice 会连接到对应的 uclass 中, uclass 用来管理同一类的驱动, 另外, 有父子关系的 udevice, 还会连接到 udevice->child_head 链表下, 方便调用。

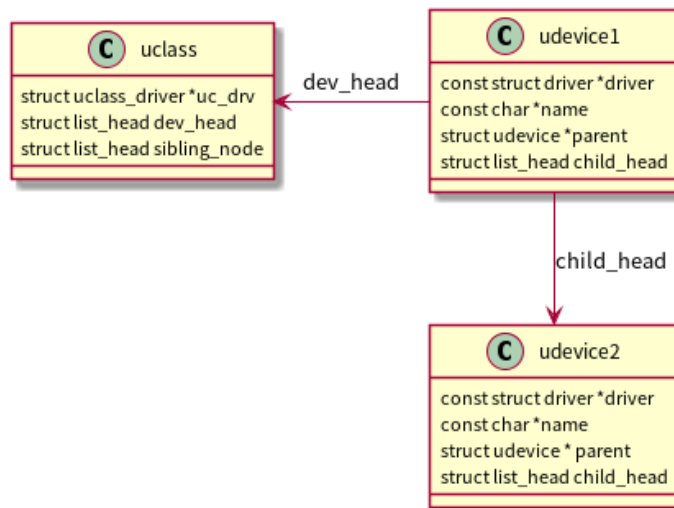


图 2: uclass 和 udevice 关系

6.2.5 driver

driver 对象, 通过 U_BOOT_DRIVER 来定义, 与 uclass_driver 有点类似, 注意区别。

```

1 struct driver {
2     char *name; // 驱动名称, dts 中的 node name
3     enum uclass_id id; // 驱动所对应的 uclass_id
4     const struct udevice_id * of_match; // 匹配函数
5     int (*bind)(struct udevice * dev); // 绑定函数
6     int (*probe)(struct udevice * dev); // 注册函数
7     int (*remove)(struct udevice * dev);
8     int (*unbind)(struct udevice * dev);
9     int (*of_to_plat)(struct udevice * dev); // 将 plat_auto 转换为本驱动的实际含义
10    int (*child_post_bind)(struct udevice * dev);
11    int (*child_pre_probe)(struct udevice * dev);
12    int (*child_post_remove)(struct udevice * dev);
13    int priv_auto;
14    int plat_auto; // 驱动需要的数据区, malloc 申请
15    int per_child_auto;
16    int per_child_plat_auto;
17    const void * ops; // driver-specific operations
18    uint32_t flags;
19    #if CONFIG_IS_ENABLED(ACPIGEN)
20        struct acpi_ops *acpi_ops;
21    #endif
22 };
  
```

- bind: 在创建 device 设备时调用;
- probe: 创建完 device 后, 由专门的 probe 阶段进行调用;

以 pinctrl 为例。

```

1 U_BOOT_DRIVER(xxx_pinctrl) = {
2     .name = "xxx_pinctrl",
  
```

```

3     .id = UCLASS_PINCTRL,
4     .of_match = xxxxxx_pinctrl_match,
5     .priv_auto_alloc_size = sizeof(struct xxx_pinctrl),
6     .ops = xxxxxx_pinctrl_ops,
7     .probe = xxxxxx_v2s_pinctrl_probe,
8     .remove = xxxxxx_v2s_pinctrl_remove,
9 };
10 //Declare a new U-Boot driver
11 #define U_BOOT_DRIVER(_name) \
12     ll_entry_declare(struct driver, _name, driver)
13
14 #define ll_entry_declare(_type, _name, _list) \
15     _type _u_boot_list_2_##_list##_2_##_name __aligned(4) \
16     __attribute__((unused, section(".u_boot_list_2_"#_list"_2_"#_name)))

```

最终转换出来如下

```

1 //同时存放在段._u_boot_list_2_driver_2_xxx_pinctrl中
2 struct driver _u_boot_list_2_driver_2_xxx_pinctrl = {
3     .name = "xxx_pinctrl",
4     .id = UCLASS_PINCTRL,
5     .of_match = xxxxxx_pinctrl_match,
6     .priv_auto_alloc_size = sizeof(struct xxx_pinctrl),
7     .ops = xxxxxx_pinctrl_ops,
8     .probe = xxxxxx_pinctrl_probe,
9     .remove = xxxxxx_pinctrl_remove,
10 }

```

同样可知，其定义之后都被存放在了段._u_boot_list_2_driver_2_xxx 中。在 u-boot.map 中可以搜索到，类似 uclass_driver，所有 driver 结构体以列表形式被放在._u_boot_list_2_driver_1 和._u_boot_list_2_driver_3 之间。综上 uclass、udevice、driver、uclass_driver 几者之间的关系如图所示。

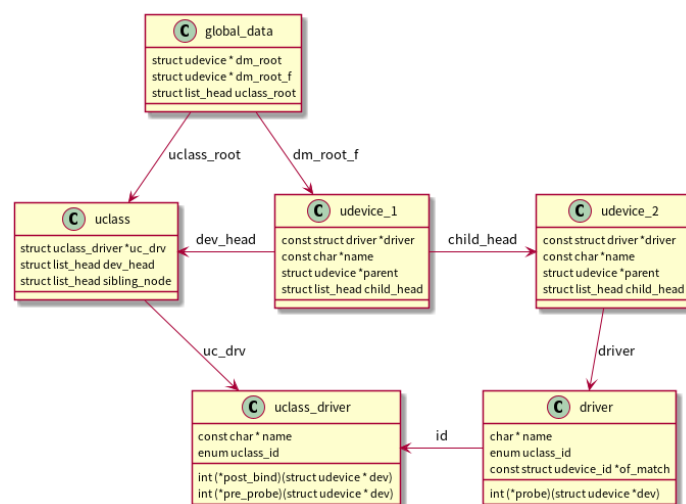


图 3: uclass/udevice/uclass_driver/driver 关系

当定义了若干的 udevice 和 driver 后，使用 uclass 和 uclass_driver 来结合 udevice 和 driver。通过这两个结构体的引入，可以将毫不相关的 udevice 是 driver 关联起来。udevice 与 driver 的绑定：通过驱动

的 `of_match` 和 `compatible` 属性来配对, 绑定。

- `uclass`: 设备组公共属性对象, 作为 `udevice` 的一个属性, 主要用来管理某个驱动类的所有的设备。
- `uclass_driver`: 设备组公共行为对象, `uclass` 的驱动程序, 主要将 `uclass` 管理的设备和驱动实现绑定、注册, 移除等操作。
- `udevice` 与 `uclass` 的绑定: `udevice` 内的 `driver` 下的 `uclass_id`, 来与 `uclass` 对应的 `uclass_driver` 的 `uclass_id` 进行匹配。
- `uclass` 与 `uclass_driver` 的绑定: 已知 `udevice` 内的 `driver` 下的 `uclass_id`, 创建 `uclass` 的同时, 通过 `uclass_id` 找到对应的 `uclass_driver` 对象, 然后将 `uclass_driver` 绑定到 `uclass` 上。

6.2.6 udevice_id

实质上就是 `compatible` 字符串, 注意和 `uclass_id` 的区别。

```
1 struct udevice_id {
2     const char * compatible;
3     ulong data;
4 };
```

6.2.7 driver_info

API `device_bind_by_name` 可以通过 `driver_info.name` 来绑定 `udevice`。

```
1 struct driver_info {
2     const char * name;
3     const void * plat;
4 #if CONFIG_IS_ENABLED(OF_PLATDATA)
5     unsigned short plat_size;
6     short parent_idx;
7 #endif
8 };
```

6.2.8 ofnode

```
1 typedef union ofnode_union {
2     struct device_node * np;
3     long of_offset;
4 } ofnode;
```

6.2.9 device_node

```
1 struct device_node {
2     const char *name;
3     const char *type;
4     phandle phandle;
5     const char *full_name;
6
7     struct property *properties;
8     struct device_node *parent;
9     struct device_node *child;
10    struct device_node * sibling;
11 };
```

6.2.10 ofprop

```

1 struct ofprop {
2     ofnode node;
3     union {
4         int offset;
5         const struct property * prop;
6     };
7 };

```

6.2.11 property

```

1 struct property {
2     char * name;
3     int length;
4     void * value;
5     struct property * next;
6 };

```

6.2.12 oftree

```

1 typedef union oftree_union {
2     struct device_node *np;
3     void * fdt;
4 } oftree;

```

6.3 重要函数

这个函数可以在 memory 中找到编译器编译进 memory 中的驱动模块信息。比如串口 serial。注册时使用名字“serial_hwd”(U_BOOT_DRIVER(serial_hwd)), 编译出来后 map 文件可以看到, 驱动会按照 __u_boot_list_2_xxx 的结构进行摆放。其中 __u_boot_list_2_driver_1 和 __u_boot_list_2_driver_3 是搜索时用的起始和结束标记, __u_boot_list_2_driver_2_xxx 就是注册的各个驱动。

使用函数 lists_driver_lookup_name(“serial_hwd”)就可以搜索到 __u_boot_list_2_driver_2_serial_hwd 地址, 进而得到这个驱动信息, 找到各驱动函数。

```

1 // u-boot.map file:
2 // __u_boot_list_2_driver_1
3 // 0x00000000fff2d730 0x0 drivers/core/dump.o
4 // __u_boot_list_2_driver_2_arm_twd_timer
5 // 0x00000000fff2d730 0x44 drivers/timer/arm_twd_timer.o
6 // 0x00000000fff2d730 _u_boot_list_2_driver_2_arm_twd_timer
7 // .....
8 // __u_boot_list_2_driver_2_root_driver
9 // 0x00000000fff2d8c8 0x44 drivers/core/root.o
10 // 0x00000000fff2d8c8 _u_boot_list_2_driver_2_root_driver
11 // __u_boot_list_2_driver_2_serial_hwd
12 // 0x00000000fff2d90c 0x44 drivers/serial/serial_hwd.o
13 // 0x00000000fff2d90c _u_boot_list_2_driver_2_serial_hwd
14 // __u_boot_list_2_driver_2_simple_bus
15 // 0x00000000fff2d950 0x44 drivers/core/simple-bus.o
16 // 0x00000000fff2d950 _u_boot_list_2_driver_2_simple_bus

```

```
17 // .....
18 // __u_boot_list_2_driver_3
19 //          0x00000000fff2d9d8          0x0 drivers/core/lists.o
20
21 U_BOOT_DRIVER(serial_hwd) = {
22     .name = "serial_hwd",
23     .id = UCLASS_SERIAL,
24     .of_match = hwd_serial_ids,
25     .of_to_plat = hwd_serial_of_to_plat,
26     .plat_auto = sizeof(struct hwd_uart_plat),
27     .probe = hwd_serial_probe,
28     .ops = &hwd_serial_ops,
29 };
30
31 struct driver *lists_driver_lookup_name(const char *name)
32 {
33     struct driver *drv = ll_entry_start(struct driver, driver);
34     const int n_ents = ll_entry_count(struct driver, driver);
35     struct driver *entry;
36
37     for (entry = drv; entry != drv + n_ents; entry++) {
38         if (!strcmp(name, entry->name))
39             return entry;
40     }
41     return NULL;
42 }
43
44
45 #define ll_entry_start(_type, _list) \
46     ({ \
47         static char start[0] __aligned(CONFIG_LINKER_LIST_ALIGN) \
48             __attribute__((unused)) \
49             __section("__u_boot_list_2_\"#_list\"_1"); \
50         _type * tmp = (_type *)&start; \
51         asm( "\"::\"+r\"(tmp);\" \
52         tmp; \
53     })
54
55 #define ll_entry_count(_type, _list) \
56     ({ \
57         _type *start = ll_entry_start(_type, _list); \
58         _type * end = ll_entry_end(_type, _list); \
59         unsigned int _ll_result = end - start; \
60         _ll_result; \
61     })
```

6.3.1 dm_scan_plat

算是一种代码中静态配置的 device，由 driver_info 配置给出，具体是扫描所有的 map 文件中的 driver_info，提取出来，再进行“device_bind_by_name”绑定设备。

```

1 static int bind_drivers_pass(struct udevice *parent, bool pre_reloc_only)
2 {
3     //找到map中固化的需要加载的driver info
4     struct driver_info *info = ll_entry_start(struct driver_info, driver_info);
5     const int n_ents = ll_entry_count(struct driver_info, driver_info);
6     bool missing_parent = false;
7     int result = 0, idx, ret;
8     //Do one iteration through the driver_info records. For of-platdata,
9     //bind only devices whose parent is already bound. If we find any
10    //device we can't bind, set missing_parent to true, which will cause
11    //this function to be called again.
12    for (idx = 0; idx < n_ents; idx++) {
13        struct udevice *par = parent, *dev;
14        const struct driver_info *entry = info + idx;
15        struct driver_rt *drt = gd_dm_driver_rt() + idx;
16        if (CONFIG_IS_ENABLED(OF_PLATDATA)) {
17            int parent_idx = driver_info_parent_id(entry);
18            if (drt->dev) continue;
19            if (CONFIG_IS_ENABLED(OF_PLATDATA_PARENT) && parent_idx != -1) {
20                struct driver_rt *parent_drt;
21                parent_drt = gd_dm_driver_rt() + parent_idx;
22                if (!parent_drt->dev) {
23                    missing_parent = true;
24                    continue;
25                }
26                par = parent_drt->dev;
27            }
28        }
29        //绑定固化的driver
30        ret = device_bind_by_name(par, pre_reloc_only, entry, &dev);
31        if (!ret) {
32            if (CONFIG_IS_ENABLED(OF_PLATDATA)) drt->dev = dev;
33        } else if (ret != -EPERM) {
34            dm_warn("No match for driver '%s'\n", entry->name);
35            if (!result || ret != -ENOENT) result = ret;
36        }
37    }
38    return result ? result : missing_parent ? -EAGAIN : 0;
39 }

```

6.3.2 dm_extended_scan

扫描 device tree。

- 第一步: 针对每个 node(从根 node / 开始)不停的递归调用 dm_scan_fdt_node 扫描每个 node 下面的 prop 属性(尤其是 compatible 属性),找到对应的设备驱动挂接;

- 第二步, 针对特定的 node 进行 dm_scan_fdt_node 扫描绑定;

- ”/chosen”
- ”/clocks”
- ”/firmware”

```
1 int dm_extended_scan(bool pre_reloc_only)
2 {
3     int ret, i;
4     const char * const nodes[] = {
5         "/chosen",
6         "/clocks",
7         "/firmware"
8     };
9
10    ret = dm_scan_fdt(pre_reloc_only);
11    if (ret) {
12        debug("dm_scan_fdt() failed: %d\n", ret);
13        return ret;
14    }
15
16    //Some nodes aren't devices themselves but may contain some
17    for (i = 0; i < ARRAY_SIZE(nodes); i++) {
18        ret = dm_scan_fdt_ofnode_path(nodes[i], pre_reloc_only);
19        if (ret) {
20            debug("dm_scan_fdt() scan for %s failed: %d\n",
21                nodes[i], ret);
22            return ret;
23        }
24    }
25    return ret;
26 }
```

6.3.3 dm_scan_other

6.3.4 dm_scan_fdt_node

该 API 可以扫描绑定入参 dev 下挂接的其他 device, 由此衍生出 3 类 API

- dm_scan_fdt: 入参是 gd->dm_root 根设备, 直接扫描整个 fdt 设备树;
- dm_scan_fdt_dev: 入参是某个 dev (类似 IIC, SPI 等总线), 可以扫描其下挂接的其他设备, 通常可以作为该类 dev 的 UCLASS_DRV 中 bind 类的挂接函数, 表示继续进行下级设备的扫描绑定;
- dm_scan_fdt_ofnode_path: 有由入参的 path 路径, 从 gd->dm_root 根设备去直接找到这个 dev 并挂接;

6.3.5 device_bind_common

这是 device 绑定的核心函数。

```
1 static int device_bind_common(struct udevice *parent, //如果是根设备, 使用gd->dm_root,
2                               //否则使用每个dts文件的设备生产的udevice
3                               const struct driver *drv, //该设备在map文件中的U_BOOT_DRIVER
4                               const char *name, //设备名字, dts文件中为node名字 (非label名)
5                               void * plat,
6                               ulong driver_data,
7                               ofnode node, //对应的dts文件node
8                               uint of_plat_size,
9                               struct udevice ** devp) //生成的device对象返回指针
10 {
11     struct udevice * dev;
12     struct uclass * uc;
13     int size, ret = 0;
14     bool auto_seq = true;
15     void *ptr;
16
17     if (CONFIG_IS_ENABLED(OFF_PLATDATA_NO_BIND)) return -ENOSYS;
18     if (devp) * devp = NULL;
19     if (!name) return -EINVAL;
20     //根据id在堆上查找已有的该类设备的uclass, 没有的话会在堆上使用
21     //uclass_add创建一个uclass, 并挂接对应的uclass_driver, 并将整个
22     //uclass挂接到gd->dm_uclass_root上
23     ret = uclass_get(drv->id, &uc);
24     if (ret) {
25         debug("Missing uclass for driver %s\n", drv->name);
26         return ret;
27     }
28     //创建设备
29     dev = calloc(1, sizeof(struct udevice));
30     if (!dev)
31         return -ENOMEM;
32     //初始化dev, 各个子元素指向dev
33     INIT_LIST_HEAD(&dev->sibling_node);
34     INIT_LIST_HEAD(&dev->child_head);
35     INIT_LIST_HEAD(&dev->uclass_node);
36 #if CONFIG_IS_ENABLED(DEVRRES)
37     INIT_LIST_HEAD(&dev->devres_head);
38 #endif
39     dev_set_plat(dev, plat);
40     dev->driver_data = driver_data;
41     dev->name = name; //设备名
42     dev_set_ofnode(dev, node); //设备node
43     dev->parent = parent; //设备的父设备
44     dev->driver = drv; //设备的驱动
45     dev->uclass = uc; //设备的uclass类
46     dev->seq_ = -1;
47     if (CONFIG_IS_ENABLED(DM_SEQ_ALIAS) &&
48         (uc->uc_drv->flags & DM_UC_FLAG_SEQ_ALIAS)) {
```



```
49          //Some devices, such as a SPI bus, I2C bus and serial ports
50          //are numbered using aliases.
51          if (CONFIG_IS_ENABLED(OF_CONTROL) &&
52              !CONFIG_IS_ENABLED(OF_PLATDATA)) {
53              if (uc->uc_drv->name && ofnode_valid(node)) {
54                  if (!dev_read_alias_seq(dev, &dev->seq_)) {
55                      auto_seq = false;
56                      log_debug("    - seq=%d\n", dev->seq_);
57                  }
58              }
59          }
60      }
61      //该uclass类下的设备可能很多，为当前设备提供一个序号seq
62      if (auto_seq && !(uc->uc_drv->flags & DM_UC_FLAG_NO_AUTO_SEQ))
63          dev->seq_ = uclass_find_next_free_seq(uc);
64      //Check if we need to allocate plat
65      if (drv->plat_auto) { //堆上申请的drv数据区域
66          bool alloc = !plat;
67          //For of-platdata, we try use the existing data, but if
68          //plat_auto is larger, we must allocate a new space
69          if (CONFIG_IS_ENABLED(OF_PLATDATA)) {
70              if (of_plat_size)
71                  dev_or_flags(dev, DM_FLAG_OF_PLATDATA);
72              if (of_plat_size < drv->plat_auto)
73                  alloc = true;
74          }
75          if (alloc) {
76              dev_or_flags(dev, DM_FLAG_ALLOC_PDATA);
77              ptr = calloc(1, drv->plat_auto);
78              if (!ptr) {
79                  ret = -ENOMEM;
80                  goto fail_alloc1;
81              }
82              //For of-platdata, copy the old plat into the new space
83              if (CONFIG_IS_ENABLED(OF_PLATDATA) && plat)
84                  memcpy(ptr, plat, of_plat_size);
85              dev_set_plat(dev, ptr); //设置数据区
86          }
87      }
88      size = uc->uc_drv->per_device_plat_auto;
89      if (size) { //uclass driver的数据区申请
90          dev_or_flags(dev, DM_FLAG_ALLOC_UCLASS_PDATA);
91          ptr = calloc(1, size);
92          if (!ptr) {
93              ret = -ENOMEM;
94              goto fail_alloc2;
95          }
96          dev_set_uclass_plat(dev, ptr); //设置数据区
```

```
97     }
98     if (parent) {
99         size = parent->driver->per_child_plat_auto; //父设备驱动为每个子设备要求的数据
100        if (!size)
101            size = parent->uclass->uc_drv->per_child_plat_auto; //父设备uclass驱动
102        if (size) {
103            dev_or_flags(dev, DM_FLAG_ALLOC_PARENT_PDATA);
104            ptr = calloc(1, size);
105            if (!ptr) {
106                ret = -ENOMEM;
107                goto fail_alloc3;
108            }
109            dev_set_parent_plat(dev, ptr); //设置数据区
110        }
111        //put dev into parent's successor list
112        list_add_tail(&dev->sibling_node, &parent->child_head);
113    }
114    //执行当前dev的parent的uclass的uclass driver中的child_post_bind接口
115    ret = uclass_bind_device(dev);
116    if (ret)
117        goto fail_uclass_bind;
118    //if we fail to bind we remove device from successors and free it
119    if (drv->bind) {
120        ret = drv->bind(dev); //执行驱动的bind接口
121        if (ret)
122            goto fail_bind;
123    }
124    if (parent && parent->driver->child_post_bind) {
125        ret = parent->driver->child_post_bind(dev); //执行父驱动的child_post_bind接口
126        if (ret)
127            goto fail_child_post_bind;
128    }
129    if (uc->uc_drv->post_bind) {
130        ret = uc->uc_drv->post_bind(dev); //执行uclass driver的post_bind接口
131        if (ret)
132            goto fail_uclass_post_bind;
133    }
134    if (parent)
135        pr_debug("Bound device %s to %s\n", dev->name, parent->name);
136    if (devp)
137        *devp = dev;
138    dev_or_flags(dev, DM_FLAG_BOUND);
139    return 0;
140 fail_uclass_post_bind:
141    // There is no child unbind() method, so no clean-up required
142 fail_child_post_bind:
143    if (CONFIG_IS_ENABLED(DM_DEVICE_REMOVE)) {
144        if (drv->unbind && drv->unbind(dev)) {
```

```
145             dm_warn("unbind() method failed on dev '%s' on error path\n",
146                     dev->name);
147         }
148     }
149 fail_bind:
150     if (CONFIG_IS_ENABLED(DM_DEVICE_REMOVE)) {
151         if (uclass_unbind_device(dev)) {
152             dm_warn("Failed to unbind dev '%s' on error path\n",
153                     dev->name);
154         }
155     }
156 fail_uclass_bind:
157     if (CONFIG_IS_ENABLED(DM_DEVICE_REMOVE)) {
158         list_del(&dev->sibling_node);
159         if (dev_get_flags(dev) & DM_FLAG_ALLOC_PARENT_PDATA) {
160             free(dev_get_parent_plat(dev));
161             dev_set_parent_plat(dev, NULL);
162         }
163     }
164 fail_alloc3:
165     if (CONFIG_IS_ENABLED(DM_DEVICE_REMOVE)) {
166         if (dev_get_flags(dev) & DM_FLAG_ALLOC_UCLASS_PDATA) {
167             free(dev_get_uclass_plat(dev));
168             dev_set_uclass_plat(dev, NULL);
169         }
170     }
171 fail_alloc2:
172     if (CONFIG_IS_ENABLED(DM_DEVICE_REMOVE)) {
173         if (dev_get_flags(dev) & DM_FLAG_ALLOC_PDATA) {
174             free(dev_get_plat(dev));
175             dev_set_plat(dev, NULL);
176         }
177     }
178 fail_alloc1:
179     devres_release_all(dev);
180     free(dev);
181     return ret;
182 }
```

6.3.6 dm_probe_devices

当 dm_init 中建立了 root 设备和 root 设备的 uclass 后，并在 dm_scan_plat 扫描建立代码中的固定 device 和 uclass 的绑定 (bind) 关系，dm_extended_scan_fdt 扫描建立 dts 中的 device 和 uclass 的绑定 (bind) 关系后，由这个 API 统一从 root 设备扫描各个设备进行 probe 操作（root 设备的 probe 操作已在 dm_init 中完成，保证 root 设备的完全可用），最终实质是调用 device_probe 进行每个 device 的 probe 操作，但是真正的被 dm_probe_devices 函数 probe 的驱动需要在驱动里面配置“DM_FLAG_PROBE_AFTER_BIND”标志才会 probe。

```
1 int device_probe(struct udevice *dev)
```

```
2 {
3     const struct driver *drv;    int ret;
4     if (!dev) return -EINVAL;
5     if (dev_get_flags(dev) & DM_FLAG_ACTIVATED) return 0;
6     drv = dev->driver;
7     ret = device_of_to_plat(dev);
8     if (ret) goto fail;
9     //Ensure all parents are probed
10    if (dev->parent) {//执行parent的probe函数
11        ret = device_probe(dev->parent);
12        if (ret) goto fail;
13        //The device might have already been probed during
14        //the call to device_probe() on its parent device
15        //(e.g. PCI bridge devices). Test the flags again
16        //so that we don't mess up the device.
17        if (dev_get_flags(dev) & DM_FLAG_ACTIVATED)
18            return 0;
19    }
20    if (CONFIG_IS_ENABLED(POWER_DOMAIN) && dev->parent &&
21        (device_get_uclass_id(dev) != UCLASS_POWER_DOMAIN) &&
22        !(drv->flags & DM_FLAG_DEFAULT_PD_CTRL_OFF)) {
23        ret = dev_power_domain_on(dev);
24        if (ret) goto fail;
25    }
26    //Process pinctrl for everything except the root device, and
27    //continue regardless of the result of pinctrl. Don't process pinctrl
28    //settings for pinctrl devices since the device may not yet be
29    //probed.
30    //This call can produce some non-intuitive results. For example, on an
31    //x86 device where dev is the main PCI bus, the pinctrl device may be
32    //child or grandchild of that bus, meaning that the child will be
33    //probed here. If the child happens to be the P2SB and the pinctrl
34    //device is a child of that, then both the pinctrl and P2SB will be
35    //probed by this call. This works because the DM_FLAG_ACTIVATED flag
36    //is set just above. However, the PCI bus' probe() method and
37    //associated uclass methods have not yet been called.
38    if (dev->parent && device_get_uclass_id(dev) != UCLASS_PINCTRL) {
39        ret = pinctrl_select_state(dev, "default");
40        if (ret && ret != -ENOSYS)
41            log_debug("Device '%s' failed to configure default pinctrl: %d (%s)
42            \n",
43                    dev->name, ret, errno_str(ret));
44    }
45    if (CONFIG_IS_ENABLED(IOMMU) && dev->parent &&
46        (device_get_uclass_id(dev) != UCLASS_IOMMU)) {
47        ret = dev_iommu_enable(dev);
48        if (ret) goto fail;
49    }
```

```
49     ret = device_get_dma_constraints(dev);
50     if (ret) goto fail;
51     ret = uclass_pre_probe_device(dev);
52     if (ret) goto fail;
53     if (dev->parent && dev->parent->driver->child_pre_probe) {
54         ret = dev->parent->driver->child_pre_probe(dev);
55         if (ret) goto fail;
56     }
57     //Only handle devices that have a valid ofnode
58     if (dev_has_ofnode(dev)) {
59         //Process 'assigned-{clocks/clock-parents/clock-rates}' properties
60         ret = clk_set_defaults(dev, CLK_DEFAULTS_PRE); //设置device的时钟
61         if (ret) goto fail;
62     }
63     if (drv->probe) { //执行device的probe
64         ret = drv->probe(dev);
65         if (ret) goto fail;
66     }
67     ret = uclass_post_probe_device(dev);
68     if (ret) goto fail_uclass;
69     if (dev->parent && device_get_uclass_id(dev) == UCLASS_PINCTRL) {
70         ret = pinctrl_select_state(dev, "default"); //设置device的IO功能
71         if (ret && ret != -ENOSYS)
72             log_debug("Device '%s' failed to configure default pinctrl: %d (%s)
73 \n",
74                         dev->name, ret, errno_str(ret));
75     }
76     return 0;
77 fail_uclass:
78     if (device_remove(dev, DM_REMOVE_NORMAL)) {
79         dm_warn("%s: Device '%s' failed to remove on error path\n",
80                 __func__, dev->name);
81     }
82 fail:
83     dev_bic_flags(dev, DM_FLAG_ACTIVATED);
84     device_free(dev);
85     return ret;
86 }
```

6.4 重要驱动

6.4.1 pinctrl

```
1 struct pinctrl_ops {
2     // @get_pins_count: Get the number of selectable pins
3     // @dev: Pinctrl device to use
4     // This function is necessary to parse the "pins" property in DTS.
5     // @Return:
6     // number of selectable named pins available in this driver
```

```
7     int (*get_pins_count)(struct udevice *dev);
8     //@get_pin_name: Get the name of a pin
9     //@dev: Pinctrl device of the pin
10    //@selector: The pin selector
11    //This function is called by the core to figure out which pin it will
12    //do operations to. This function is necessary to parse the "pins"
13    //property in DTS.
14    //@Return: const pointer to the name of the pin
15    const char * (* get_pin_name)(struct udevice * dev, unsigned selector);
16    //@get_groups_count: Get the number of selectable groups
17    //@dev: Pinctrl device to use
18    //This function is necessary to parse the "groups" property in DTS.
19    //@Return:
20    // number of selectable named groups available in the driver
21    int (*get_groups_count)(struct udevice *dev);
22    //@get_group_name: Get the name of a group
23    //@dev: Pinctrl device of the group
24    //@selector: The group selector
25    //This function is called by the core to figure out which group it
26    //will do operations to. This function is necessary to parse the
27    //"groups" property in DTS.
28    //@Return: Pointer to the name of the group
29    const char *(* get_group_name)(struct udevice * dev, unsigned selector);
30    //@get_functions_count: Get the number of selectable functions
31    //@dev: Pinctrl device to use
32    //This function is necessary for pin-muxing.
33    //@Return:
34    // number of selectable named functions available in this driver
35    int (*get_functions_count)(struct udevice *dev);
36    //@get_function_name: Get the name of a function
37    //@dev: Pinmux device of the function
38    //@selector: The function selector
39    //This function is called by the core to figure out which mux setting
40    //it will map a certain device to. This function is necessary for
41    //pin-muxing.
42    //@Return:
43    // Pointer to the function name of the muxing selector
44    const char *(* get_function_name)(struct udevice * dev,unsigned selector);
45    //@pinmux_set: Mux a pin to a function
46    //@dev: Pinctrl device to use
47    //@pin_selector: The pin selector
48    //@func_selector: The func selector
49    //On simple controllers one of @pin_selector or @func_selector may be
50    //ignored. This function is necessary for pin-muxing against a single pin.
51    //@Return: 0 if OK, or negative error code on failure
52    int (*pinmux_set)(struct udevice * dev, unsigned pin_selector, unsigned func_selector);
53    //@pinmux_group_set: Mux a group of pins to a function
54    //@dev: Pinctrl device to use
```

```
55 // @group_selector: The group selector
56 // @func_selector: The func selector
57 // On simple controllers one of @group_selector or @func_selector may be
58 // ignored. This function is necessary for pin-muxing against a group of pins.
59 // @Return: 0 if OK, or negative error code on failure
60 int (*pinmux_group_set)(struct udevice * dev, unsigned group_selector, unsigned func_selector);
61 // @pinmux_property_set: Enable a pinmux group
62 // @dev: Pinctrl device to use
63 // @pinmux_group: A u32 representing the pin identifier and mux
64 // settings. The exact format of a pinmux group is left
65 // up to the driver.
66 // Mux a single pin to a single function based on a driver-specific
67 // pinmux group. This function is necessary for parsing the "pinmux"
68 // property in DTS, and for pin-muxing against a pinmux group.
69 // @Return:
70 // Pin selector for the muxed pin if OK, or negative error code on
71 // failure
72 int (*pinmux_property_set)(struct udevice *dev, u32 pinmux_group);
73 // @pinconf_num_params:
74 // Number of driver-specific parameters to be parsed from device
75 // trees. This member is necessary for pin configuration.
76 unsigned int pinconf_num_params;
77
78 // @pinconf_params:
79 // List of driver-specific parameters to be parsed from the device
80 // tree. This member is necessary for pin configuration.
81 const struct pinconf_param * pinconf_params;
82 // @pinconf_set: Configure an individual pin with a parameter
83 // @dev: Pinctrl device to use
84 // @pin_selector: The pin selector
85 // @param: An &enum pin_config_param from @pinconf_params
86 // @argument: The argument to this param from the device tree, or
87 // @pinconf_params.default_value
88 // This function is necessary for pin configuration against a single pin.
89 // @Return: 0 if OK, or negative error code on failure
90 int (*pinconf_set)(struct udevice * dev, unsigned pin_selector,
91 unsigned param, unsigned argument);
92 // @pinconf_group_set: Configure all pins in a group with a parameter
93 // @dev: Pinctrl device to use
94 // @pin_selector: The group selector
95 // @param: A &enum pin_config_param from
96 // @pinconf_params
97 // @argument: The argument to this param from the device tree, or
98 // @pinconf_params.default_value
99 // This function is necessary for pin configuration against a group of pins.
100 // @Return: 0 if OK, or negative error code on failure
101 int (*pinconf_group_set)(struct udevice * dev, unsigned group_selector,
102 unsigned param, unsigned argument);
```

```
103    // @set_state: Configure a pinctrl device
104    // @dev: Pinctrl device to use
105    // @config: Pseudo device pointing a config node
106    // This function is required to be implemented by all pinctrl drivers.
107    // Drivers may set this member to pinctrl_generic_set_state(), which
108    // will call other functions in &struct pinctrl_ops to parse
109    // @config.
110    // @Return: 0 if OK, or negative error code on failure
111    int (*set_state)(struct udevice *dev, struct udevice * config);
112    // @set_state_simple: Configure a pinctrl device
113    // @dev: Pinctrl device to use
114    // @config: Pseudo-device pointing a config node
115    // This function is usually a simpler version of set_state(). Only the
116    // first pinctrl device on the system is supported by this function.
117    // @Return: 0 if OK, or negative error code on failure
118    int (*set_state_simple)(struct udevice *dev, struct udevice * periph);
119    // @request: Request a particular pinctrl function
120    // @dev: Device to adjust (%UCLASS_PINCTRL)
121    // @func: Function number (driver-specific)
122    // This activates the selected function.
123    // @Return: 0 if OK, or negative error code on failure
124    int (*request)(struct udevice * dev, int func, int flags);
125    // @get_periph_id: Get the peripheral ID for a device
126    // @dev: Pinctrl device to use for decoding
127    // @periph: Device to check
128    // This generally looks at the peripheral's device tree node to work
129    // out the peripheral ID. The return value is normally interpreted as
130    // &enum periph_id. so long as this is defined by the platform (which it
131    // should be).
132    // @Return:
133    // Peripheral ID of @periph, or %-ENOENT on error
134    int (*get_periph_id)(struct udevice *dev, struct udevice * periph);
135    // @get_gpio_mux: Get the mux value for a particular GPIO
136    // @dev: Pinctrl device to use
137    // @banknum: GPIO bank number
138    // @index: GPIO index within the bank
139    // This allows the raw mux value for a GPIO to be obtained. It is
140    // useful for displaying the function being used by that GPIO, such
141    // as with the 'gpio' command. This function is internal to the GPIO
142    // subsystem and should not be used by generic code. Typically it is
143    // used by a GPIO driver with knowledge of the SoC pinctrl setup.
144    // @Return: Mux value (SoC-specific, e.g. 0 for input, 1 for output)
145    int (*get_gpio_mux)(struct udevice * dev, int banknum, int index);
146    // @get_pin_muxing: Show pin muxing
147    // @dev: Pinctrl device to use
148    // @selector: Pin selector
149    // @buf: Buffer to fill with pin muxing description
150    // @size: Size of @buf
```



```

151 //This allows to display the muxing of a given pin. It's useful for
152 //debug purposes to know if a pin is configured as GPIO or as an
153 //alternate function and which one. Typically it is used by a PINCTRL
154 //driver with knowledge of the SoC pinctrl setup.
155 //Return: 0 if OK, or negative error code on failure
156 int (*get_pin_muxing)(struct udevice * dev, unsigned int selector,
157                      char * buf, int size);
158 //gpio_request_enable: Request and enable GPIO on a certain pin.
159 //dev: Pinctrl device to use
160 //selector: Pin selector
161 //Implement this only if you can mux every pin individually as GPIO.
162 //The affected GPIO range is passed along with an offset(pin number)
163 //into that specific GPIO range - function selectors and pin groups are
164 //orthogonal to this, the core will however make sure the pins do not
165 //collide.
166 //Return: 0 if OK, or negative error code on failure
167 int (*gpio_request_enable)(struct udevice * dev, unsigned int selector);
168 //gpio_disable_free: Free up GPIO muxing on a certain pin.
169 //dev: Pinctrl device to use
170 //selector: Pin selector
171 //This function is the reverse of gpio_request_enable.
172 //Return: 0 if OK, or negative error code on failure
173 int (*gpio_disable_free)(struct udevice *dev, unsigned int selector);
174 };
175

```

7 环境变量(Environment)

- 默认环境变量默认的环境变量如下。

```

1 static char default_environment[] =
2 {
3     ENV_CALLBACK_VAR "=" CONFIG_ENV_CALLBACK_LIST_DEFAULT "\0"
4     ENV_FLAGS_VAR "=" CONFIG_ENV_FLAGS_LIST_DEFAULT "\0"
5     "bootargs=" CONFIG_BOOTARGS "\0"
6     "bootcmd=" CONFIG_BOOTCOMMAND "\0"
7     "bootdelay=" __stringify(CONFIG_BOOTDEL) "\0"
8     "baudrate=" __stringify(CONFIG_BAUDRAT) "\0"
9     "loads_echo=" __stringify(CONFIG_LOADS_E) "\0"
10    "ethprime=" CONFIG_ETHPRIME "\0"
11    "ipaddr=" CONFIG_IPADDR "\0"
12    "serverip=" CONFIG_SERVERIP "\0"
13    "autoload=0" "\0"
14    "preboot=" CONFIG_PREBOOT "\0"
15    "rootpath=" CONFIG_ROOTPATH "\0"
16    "gatewayip=" CONFIG_GATEWAYIP "\0"
17    "netmask=" CONFIG_NETMASK "\0"
18    "hostname=" CONFIG_HOSTNAME "\0"

```

```

19     "bootfile="                CONFIG_BOOTFILE                "\0"
20     "loadaddr="                __stringify(CONFIG_SYS_LOAD_ADDR) "\0"
21     "pcideLAY="                __stringify(CONFIG_PCI_BOOTDELAY) "\0"
22     "arch="                    CONFIG_SYS_ARCH                "\0"
23     "cpu="                    CONFIG_SYS_CPU                "\0"
24     "board="                   CONFIG_SYS_BOARD                "\0"
25     "board_name="              CONFIG_SYS_BOARD                "\0"
26     "vendor="                  CONFIG_SYS_VENDOR                "\0"
27     "soc="                     CONFIG_SYS_SOC                "\0"
28     "env_fdt_path="            CONFIG_ENV_FDT_PATH                "\0"
29     "bootlimit="              __stringify(CONFIG_BOOTCOUNT_BOOTLIMIT) "\0"
30     "mtdids="                  CONFIG_MTDIDS_DEFAULT                "\0"
31     "mtdparts="                CONFIG_MTDPARTS_DEFAULT                "\0"
32     CONFIG_EXTRA_ENV_TEXT //This is created in the Makefile
33     CFG_EXTRA_ENV_SETTINGS                "\0"
34 }

```

- 环境变量存储位置

```

1  enum env_location {
2      ENVL_UNKNOWN,
3      ENVL_EEPROM,
4      ENVL_EXT4,
5      ENVL_FAT,
6      ENVL_FLASH,
7      ENVL_MMC,
8      ENVL_NAND,
9      ENVL_NVRAM,
10     ENVL_ONENAND,
11     ENVL_REMOTE,
12     ENVL_SPI_FLASH,
13     ENVL_UBI,
14     ENVL_NOWHERE,
15     ENVL_COUNT,
16 };

```

- 环境变量驱动结构体

```

1  struct env_driver {
2      const char * name;
3      enum env_location location;
4
5      //load() - Load the environment from storage
6      //This method is required for loading environment
7      //@return 0 if OK, -ve on error
8      int (* load)(void);
9
10     //save() - Save the environment to storage
11     //This method is required for 'saveenv' to work.

```

```
12         //@return 0 if OK, -ve on error
13         int (* save)(void);
14
15         //erase() - Erase the environment on storage
16         //This method is optional and required for 'eraseenv' to work.
17         //@return 0 if OK, -ve on error
18         int (* erase)(void);
19
20         //init() - Set up the initial pre-relocation environment
21         //This method is optional.
22         //@return 0 if OK, -ENOENT if no initial environment could be found,
23         //other -ve on error
24         int (* init)(void);
25     };
26     //驱动声明
27     #define U_BOOT_ENV_LOCATION(_name)      \
28         ll_entry_declare(struct env_driver, _name, env_driver)
```

- 环境变量操作

```
1  enum env_operation {
2      ENVOP_GET_CHAR, // we want to call the get_char function
3      ENVOP_INIT, // we want to call the init function
4      ENVOP_LOAD, // we want to call the load function
5      ENVOP_SAVE, // we want to call the save function
6      ENVOP_ERASE, // we want to call the erase function
7  };
```

8 命令(CMD)

8.1 重要结构

每个命令定义时采用的结构如下。

```
1  struct cmd_tbl {
2      char * name; // Command Name
3      int maxargs; // maximum number of arguments
4      //Same as ->cmd() except the command
5      //tells us if it can be repeated.
6      //Replaces the old ->repeatable field
7      //which was not able to make
8      //repeatable property different for
9      //the main command and sub-commands.
10     int (* cmd_rep)(struct cmd_tbl *cmd,
11                     int flags,
12                     int argc,
13                     char *const argv[],
14                     int * repeatable);
15     // Implementation function
```

```

16     int (* cmd)(struct cmd_tbl *cmd,
17                 int flags,
18                 int argc,
19                 char *const argv[]);
20     char *usage; // Usage message (short)
21 #ifdef CONFIG_SYS_LONGHELP
22     const char * help; // Help message (long)
23 #endif
24 #ifdef CONFIG_AUTO_COMPLETE
25     // do auto completion on the arguments
26     int (* complete)(int argc,
27                     char *const argv[],
28                     char last_char,
29                     int maxv,
30                     char * cmdv[]);
31 #endif
32 };

```

8.2 命令定义

通过使用宏 U_BOOT_CMD 定义如下,最终定义出 map 文件中列的符号如”__u_boot_list_2_cmd_2_xxx”的命令变量。

- name: 命令名字;
- _maxargs: 最大参数数量;
- _rep: 是否可以重复;
- _cmd: 具体的命令执行函数;
- _usage: 命令帮助简介;
- _help: 命令详细帮助;

```

1 // u-boot.map file:
2 // __u_boot_list_2_cmd_1      0x00000000fff2ba34  0x0 common/command.o
3 // __u_boot_list_2_cmd_2_binfo 0x00000000fff2ba4c  _u_boot_list_2_cmd_2_binfo
4 // ...
5 // __u_boot_list_2_cmd_2_version 0x00000000fff2bc8c  _u_boot_list_2_cmd_2_version
6 // __u_boot_list_2_cmd_3      0x00000000fff2bca4  0x0 cmd/help.o
7 // include "command.h"
8 #define U_BOOT_CMD(_name, _maxargs, _rep, _cmd, _usage, _help) \
9     U_BOOT_CMD_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help, NULL)
10
11 #define U_BOOT_CMD_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help, _comp) \
12     ll_entry_declare(struct cmd_tbl, _name, cmd) = \
13         U_BOOT_CMD_MKENT_COMPLETE(_name, _maxargs, _rep, _cmd, \
14                                     _usage, _help, _comp);
15
16 #define U_BOOT_CMD_MKENT_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help, _comp) \
17     { #_name, _maxargs, \

```

```

18         _rep ? cmd_always_repeatable : cmd_never_repeatable, \
19         _cmd, _usage, _CMD_HELP(_help) _CMD_COMPLETE(_comp) }
20 # define _CMD_HELP(x) x,
21 # define _CMD_COMPLETE(x) x,
22 //具体命令定义"bdfinfo"
23 U_BOOT_CMD(bdfinfo, 1, 1, do_bdfinfo,
24         "print Board Info structure",
25         "");

```

8.3 命令使用

在使用时,遵循如下步骤

- 通过 `getc` 函数不停的收集串口输入的命令字符,并存储于缓存的 `buffer` 中;
- 解析 `buffer` 中的每个字符,剔除非法字符,转换特殊符号,分割多个命令,获取到每个命令的名字 `name`;
- 通过如下的 `find_cmd`,将符号表中的命令变量逐一进行 `name` 字段对比,找到 `cmd`;
- 使用 `cmd` 的 `cmd_rep` 回调调用该命令的具体执行;

```

1 struct cmd_tbl * find_cmd_tbl(const char * cmd, struct cmd_tbl * table,int table_len)
2 {
3 #ifdef CONFIG_CMDLINE
4     struct cmd_tbl * cmdtp;
5     struct cmd_tbl * cmdtp_temp = table; // Init value
6     const char * p;
7     int len;
8     int n_found = 0;
9
10    if (!cmd)
11        return NULL;
12    //Some commands allow length modifiers (like "cp.b");
13    //compare command name only until first dot.
14    len = ((p = strchr(cmd, '.')) == NULL) ? strlen (cmd) : (p - cmd);
15
16    for (cmdtp = table; cmdtp != table + table_len; cmdtp++) {
17        if (strncmp(cmd, cmdtp->name, len) == 0) {
18            if (len == strlen(cmdtp->name))
19                return cmdtp; // full match
20            cmdtp_temp = cmdtp; // abbreviated command ?
21            n_found++;
22        }
23    }
24    if (n_found == 1) { // exactly one match
25        return cmdtp_temp;
26    }
27 #endif
28    return NULL; // not found or ambiguous command
29 }

```

```
30
31 struct cmd_tbl * find_cmd(const char * cmd)
32 {
33     struct cmd_tbl * start = ll_entry_start(struct cmd_tbl, cmd);
34     const int len = ll_entry_count(struct cmd_tbl, cmd);
35     return find_cmd_tbl(cmd, start, len);
36 }
37
38 static int cmd_call(struct cmd_tbl *cmdtp, int flag, int argc,
39                     char *const argv[], int * repeatable)
40 {
41     int result;
42     result = cmdtp->cmd_rep(cmdtp, flag, argc, argv, repeatable);
43     if (result)
44         debug("Command failed, result=%d\n", result);
45     return result;
46 }
```

8.4 注意事项

- 多条命令使用“;”分割;

8.5 重要命令

8.5.1 bootmenu

8.5.2 bootz

对于 ARM 来讲,可以通过命令来启动内核 `bootz kernel_addr initrd_address dtb_address`,

- `dtb_address`: 作为 `bootz` 或者 `bootm` 的最后一个参数;
- `kernel_addr`: 为内核映像的地址;
- `initrd_address`: 第一个参数为 `initrd` 的地址,若不存在 `initrd`,可以 “-” 符号代替。Linux 初始 RAM 磁盘 (`initrd`) 是在系统引导过程中挂载的一个临时根文件系统,来支持两阶段的引导过程。`initrd` 文件中包含了各种可执行程序 and 驱动程序,它们可以用来挂载实际的根文件系统,然后再将这个 `initrd` RAM 磁盘卸载,并释放内存。在很多嵌入式 Linux 系统中,`initrd` 就是最终的根文件系统。

8.5.3 cp

将源地址数据拷贝到目的地址,支持拷贝进入 flash。

8.5.4 env

执行环境变量里面对应的命令

```
1 static int do_env(struct cmd_tbl *cmdtp, int flag, int argc, char *const argv[])
2 {
3     struct cmd_tbl * cp;
4     if (argc < 2) return CMD_RET_USAGE;
5     //drop initial "env" arg
```

```
6     argc--;
7     argv++;
8     cp = find_cmd_tbl(argv[0], cmd_env_sub, ARRAY_SIZE(cmd_env_sub));
9     if (cp) return cp->cmd(cmdtp, flag, argc, argv);
10    return CMD_RET_USAGE;
11 }
```

8.5.5 go

go 命令用于运行在参数中给定的 address 函数。

```
1 unsigned long do_go_exec(ulong (*entry)(int, char * const []), int argc,
2                          char *const argv[])
3 {
4     return entry (argc, argv);
5 }
6
7 static int do_go(struct cmd_tbl *cmdtp, int flag, int argc, char *const argv[])
8 {
9     ulong addr, rc;
10    int rcode = 0;
11    if (argc < 2) return CMD_RET_USAGE;
12    addr = hextoul(argv[1], NULL);
13    printf ("## Starting application at 0x%08lX ...\n", addr);
14    flush();
15    //pass address parameter as argv[0] (aka command name),
16    //and all remaining args
17    rc = do_go_exec ((void * )addr, argc - 1, argv + 1);
18    if (rc != 0) rcode = 1;
19    printf ("## Application terminated, rc = 0x%lX\n", rc);
20    return rcode;
21 }
22
```

8.5.6 loadx/y/b

通过串口,使用 xmode/ymode/kermit 协议把数据下载到 memory 中。

8.5.7 loop

8.5.8 md

8.5.9 mm

8.5.10 printenv

8.5.11 reset

reset 命令清理完环境后,做 cpu 复位,其中 reset_cpu 函数可以用户自定义。

```
1 int do_reset(struct cmd_tbl *cmdtp, int flag, int argc, char * const argv[])
2 {
3     puts ("resetting ...\n");
```

```
4     flush();
5     disable_interrupts();
6     reset_misc();
7     reset_cpu();
8     /*NOTREACHED*/
9     return 0;
10 }
```

8.5.12 run

run 命令用于运行 command, 该 command 存于环境变量中。

```
1 int do_run(struct cmd_tbl *cmdtp, int flag, int argc, char *const argv[])
2 {
3     int i, ret;
4     if (argc < 2) return CMD_RET_USAGE;
5     for (i = 1; i < argc; ++i) {
6         char * arg;
7         arg = env_get(argv[i]);
8         if (arg == NULL) {
9             printf("## Error: \"%s\" not defined\n", argv[i]);
10            return 1;
11        }
12        ret = run_command(arg, flag | CMD_FLAG_ENV);
13        if (ret) return ret;
14    }
15    return 0;
16 }
```

8.5.13 setenv

8.5.14 fdt

- addr: 设置 dtb 的地址, fdt addr 0xXXX;
- print: 输出 dtb 的 dts 格式, fdt print;

8.5.15 dm

dm 命令可以很直观的画出 dts 设备的挂接关系。dm tree

9 事件(Event)

事件 Event, 用于在某些阶段, 需要做一些回调处理, 通过 event_register 挂接对应的处理函数。并在发生该阶段事件的地方使用 event_notify_null 来触发调用该事件的执行函数。系统默认的事件类型如 enum event_t。事件机制用的较少。

9.1 事件注册

注意事件是注册在堆上, 并挂接到 gd->event_state 上。


```
1  enum event_t {
2      EVT_NONE,
3      EVT_TEST,
4      //Events related to driver model
5      EVT_DM_POST_INIT_F,
6      EVT_DM_PRE_PROBE,
7      EVT_DM_POST_PROBE,
8      EVT_DM_PRE_REMOVE,
9      EVT_DM_POST_REMOVE,
10     //Init hooks
11     EVT_MISC_INIT_F,
12     //Device tree fixups before booting
13     EVT_FT_FIXUP,
14     //To be called once, before calling main_loop()
15     EVT_MAIN_LOOP,
16     EVT_COUNT
17 };
18
19 int event_register(const char *id, enum event_t type, event_handler_t func, void *ctx)
20 {
21     struct event_state *state = gd_event_state();
22     struct event_spy * spy;
23     spy = malloc(sizeof(* spy));
24     if (!spy)
25         return log_msg_ret("alloc", -ENOMEM);
26     spy->id = id;
27     spy->type = type;
28     spy->func = func;
29     spy->ctx = ctx;
30     list_add_tail(&spy->sibling_node, &state->spy_head);
31     return 0;
32 }
```

9.2 事件调用

```
1  int event_notify_null(enum event_t type)
2  {
3      return event_notify(type, NULL, 0);
4  }
5
6  int event_notify(enum event_t type, void *data, int size)
7  {
8      struct event event;
9      int ret;
10     event.type = type;
11     if (size > sizeof(event.data))
12         return log_msg_ret("size", -E2BIG);
13     memcpy(&event.data, data, size);
14     ret = notify_static(&event);
```

```
15     if (ret)
16         return log_msg_ret("sta", ret);
17     if (CONFIG_IS_ENABLED(EVENT_DYNAMIC)) {
18         ret = notify_dynamic(&event);
19         if (ret)
20             return log_msg_ret("dyn", ret);
21     }
22     return 0;
23 }
24
25 static int notify_static(struct event *ev)
26 {
27     struct evspy_info *start =
28         ll_entry_start(struct evspy_info, evspy_info);
29     const int n_ents = ll_entry_count(struct evspy_info, evspy_info);
30     struct evspy_info *spy;
31     for (spy = start; spy != start + n_ents; spy++) {
32         if (spy->type == ev->type) {
33             int ret;
34             log_debug("Sending event %x/%s to spy '%s'\n", ev->type,
35                     event_type_name(ev->type), event_spy_id(spy));
36             ret = spy->func(NULL, ev);
37             //TODO: Handle various return codes to
38             //- claim an event (no others will see it)
39             //- return an error from the event
40             if (ret)
41                 return log_msg_ret("spy", ret);
42         }
43     }
44     return 0;
45 }
46
47 static int notify_dynamic(struct event *ev)
48 {
49     struct event_state *state = gd_event_state();
50     struct event_spy *spy, *next;
51     list_for_each_entry_safe(spy, next, &state->spy_head, sibling_node) {
52         if (spy->type == ev->type) {
53             int ret;
54             log_debug("Sending event %x/%s to spy '%s'\n", ev->type,
55                     event_type_name(ev->type), spy->id);
56             ret = spy->func(spy->ctx, ev);
57             //TODO: Handle various return codes to
58             //- claim an event (no others will see it)
59             //- return an error from the event
60             if (ret)
61                 return log_msg_ret("spy", ret);
62         }
63     }
```

```
63     }  
64     return 0;  
65 }
```

10 Trace

Trace 和 Stage 有点类似。但是 Trace, 可以通过在运行时修改变量"trace_enabled", 进行 trace 打开, 进而调试。u-boot 实际用的较少。

```
1 static char trace_enabled __section(".data");  
2 static char trace_initd __section(".data");
```

11 Stage

11.1 用法

- 正常记录: 当进入 u-boot 某个关键流程阶段时, 可以使用 bootstage_mark 记录函数名;

- 函数记录: #define bootstage_mark(id) bootstage_mark_name(id, __func__)
- 特定记录: 当进入某些特定环节, 做特定记录

```
1 bootstage_mark_name(BOOTSTAGE_ID_BOOTM_HANDOFF, "start_kernel");  
2 bootstage_mark_name(BOOTSTAGE_ID_END_TPL, "end tpl");  
3 bootstage_mark_name(BOOTSTAGE_ID_BOOTM_START, "bootm_start");  
4 bootstage_mark_name(BOOTSTAGE_KERNELREAD_START, "tftp_start");  
5 bootstage_mark_name(BOOTSTAGE_KERNELREAD_STOP, "tftp_done");  
6 bootstage_mark_name(BOOTSTAGE_ID_USB_START, "usb_start");  
7 bootstage_mark_name(BOOTSTAGE_ID_START_UBOOT_F, "board_init_f");  
8 bootstage_mark_name(BOOTSTAGE_ID_START_UBOOT_R, "board_init_r");  
9 bootstage_mark_name(BOOTSTAGE_ID_MAIN_LOOP, "main_loop");  
10 bootstage_mark_name(get_bootstage_id(false), "end phase");  
11 bootstage_mark_name(BOOTSTAGE_ID_BOOTP_STOP, "bootp_stop");  
12 bootstage_mark_name(BOOTSTAGE_ID_BOOTP_START, "bootp_start");  
13 bootstage_mark_name(BOOTSTAGE_ID_ETH_START, "eth_start");
```

- 错误记录: 当进入某些异常流程阶段是, 记录函数名

```
1 bootstage_error(BOOTSTAGE_ID_FIT_TYPE);  
2 bootstage_error(BOOTSTAGE_ID_FIT_COMPRESSION);  
3 bootstage_error(BOOTSTAGE_ID_FIT_OS);  
4 bootstage_error(BOOTSTAGE_ID_FIT_LOADADDR);  
5 bootstage_error(BOOTSTAGE_ID_DECOMP_IMAGE);  
6 bootstage_error(BOOTSTAGE_ID_OVERWRITTEN);  
7 bootstage_error(BOOTSTAGE_ID_CHECK_BOOT_OS);  
8 bootstage_error(BOOTSTAGE_ID_DECOMP_UNIMPL);  
9 bootstage_error(BOOTSTAGE_ID_CHECK_MAGIC);  
10 bootstage_error(BOOTSTAGE_ID_CHECK_HEADER);  
11 bootstage_error(BOOTSTAGE_ID_CHECK_CHECKSUM);  
12 bootstage_error(BOOTSTAGE_ID_CHECK_ARCH);  
13 bootstage_error(BOOTSTAGE_ID_CHECK_IMAGETYPE);
```

```

14 bootstage_error(BOOTSTAGE_ID_FIT_KERNEL_INFO);
15 bootstage_error(BOOTSTAGE_ID_BOOT_OS_RETURNED);
16 bootstage_error(BOOTSTAGE_ID_RD_MAGIC);
17 bootstage_error(BOOTSTAGE_ID_RD_HDR_CHECKSUM);
18 bootstage_error(BOOTSTAGE_ID_RD_CHECKSUM);
19 bootstage_error(BOOTSTAGE_ID_RAMDISK);
20 bootstage_error(bootstage_id + BOOTSTAGE_SUB_FORMAT);
21 bootstage_error(bootstage_id + BOOTSTAGE_SUB_SUBNODE);
22 bootstage_error(bootstage_id + BOOTSTAGE_SUB_HASH);
23 bootstage_error(bootstage_id + BOOTSTAGE_SUB_CHECK_ARCH);
24 bootstage_error(bootstage_id + BOOTSTAGE_SUB_CHECK_ALL);
25 bootstage_error(bootstage_id + BOOTSTAGE_SUB_GET_DATA);
26 bootstage_error(bootstage_id + BOOTSTAGE_SUB_LOAD);
27 bootstage_error(BOOTSTAGE_ID_IDE_ADDR);
28 bootstage_error(BOOTSTAGE_ID_IDE_TYPE);
29 bootstage_error(BOOTSTAGE_ID_IDE_PART_READ);
30 bootstage_error(BOOTSTAGE_ID_IDE_CHECKSUM);
31 bootstage_error(BOOTSTAGE_ID_IDE_FORMAT);
32 bootstage_error(BOOTSTAGE_ID_IDE_READ);
33 bootstage_error(BOOTSTAGE_ID_IDE_FIT_READ);
34 bootstage_error(BOOTSTAGE_ID_NAND_SUFFIX);
35 bootstage_error(BOOTSTAGE_ID_NAND_HDR_READ);
36 bootstage_error(BOOTSTAGE_ID_NAND_TYPE);
37 bootstage_error(BOOTSTAGE_ID_NAND_READ);
38 bootstage_error(BOOTSTAGE_ID_NAND_FIT_READ);
39 bootstage_error(BOOTSTAGE_ID_NAND_SUFFIX);
40 bootstage_error(BOOTSTAGE_ID_NAND_BOOT_DEVICE);
41 bootstage_error(BOOTSTAGE_ID_NAND_AVAILABLE);
42 bootstage_error(BOOTSTAGE_ID_NET_START);
43 bootstage_error(BOOTSTAGE_ID_NET_NETLOOP_OK);
44 bootstage_error(BOOTSTAGE_ID_NET_LOADED);
45 bootstage_error(BOOTSTAGE_ID_NET_DONE_ERR);
46 bootstage_error(BOOTSTAGE_ID_NET_CHECKSUM);
47 bootstage_error(BOOTSTAGE_ID_NEED_RESET);
48 bootstage_error(BOOTSTAGE_ID_NET_ETH_START);
49 bootstage_error(BOOTSTAGE_ID_POST_FAIL_R);

```

实际使用时,通过导出内存? 还是 u-boot 命令输出内存调测?

11.2 原理

由 struct bootstage_data 做整个记录的结构, 该结构由 malloc 申请内存并挂在 gd 上, 每个记录记载在 struct bootstage_record 里面, 最后识别出 u-boot 的若干关键阶段并用 enum bootstage_id 作为 ID 记号, 每次记录时, 如果有相同的 ID 存在, 则不再记录, 否则找到一个 recode, 记录当前的时间戳"time_us", 名"name", 以及 ID。

```

1 enum {
2     RECORD_COUNT = CONFIG_VAL(BOOTSTAGE_RECORD_COUNT),
3 };

```

```
4
5 struct bootstage_record {
6     ulong time_us;
7     uint32_t start_us;
8     const char * name;
9     int flags; // see enum bootstage_flags
10    enum bootstage_id id;
11 };
12
13 struct bootstage_data {
14     uint rec_count;
15     uint next_id;
16     struct bootstage_record record[RECORD_COUNT];
17 };
18
```

12 编译

12.1 u_boot_list

12.1.1 u_boot_list_2 类型

u-boot 可以通过 ll_entry_declare 定义如下的数据结构变量,并存放在 section 中 __u_boot_list_2_#_list_2_#_name。

- ll_entry_declare(struct udevice, _name, udevice);
 - DM_DEVICE_INST
- ll_entry_declare(struct driver, _name, driver);
 - U_BOOT_DRIVER
- ll_entry_declare(struct driver_info, _name, driver_info);
 - U_BOOT_DRVINFO
- ll_entry_declare(struct uclass_driver, _name, uclass_driver);
 - UCLASS_DRIVER
- ll_entry_declare(struct uclass, _name, uclass);
 - DM_UCLASS_INST
- ll_entry_declare(struct sandbox_state_io, _name, state_io);
 - SANDBOX_STATE_IO
- ll_entry_declare(struct blk_driver, _name, blk_driver);
 - U_BOOT_LEGACY_BLK
- ll_entry_declare(struct bootdev_hunter, _name, bootdev_hunter);
 - BOOTDEV_HUNTER
- ll_entry_declare(struct cmd_tbl, _name, cmd);

- U_BOOT_CMD_COMPLETE
 - U_BOOT_CMDREP_COMPLETE
- ll_entry_declare(struct acpi_writer, _name, acpi_writer);
 - ACPI_WRITER
- ll_entry_declare(struct efi_unit_test, _name, efi_unit_test);
 - EFI_UNIT_TEST
- ll_entry_declare(struct env_clbk_tbl, name, env_clbk)
 - U_BOOT_ENV_CALLBACK
- ll_entry_declare(struct env_driver, _name, env_driver);
 - U_BOOT_ENV_LOCATION
- ll_entry_declare(struct evspy_info, _type##_func, ;
 - EVENT_SPY
- ll_entry_declare(struct g_dnl_bind_callback, ;
 - DECLARE_GADGET_BIND_CALLBACK
- ll_entry_declare(struct i2c_adapter, _name, i2c) ;
 - U_BOOT_I2C_ADAP_COMPLETE
- ll_entry_declare(struct crypto_algo, _name, cryptos);
 - U_BOOT_CRYPTO_ALGO
- ll_entry_declare(struct padding_algo, _name, paddings);
 - U_BOOT_PADDING_ALGO
- ll_entry_declare(struct fit_loadable_tbl, _function, fit_loadable);
 - U_BOOT_FIT_LOADABLE_HANDLER
- ll_entry_declare(struct arm_smccc_feature, _name, arm_smccc_feature);
 - ARM_SMCCC_FEATURE_DRIVER
- ll_entry_declare(struct log_driver, _name, log_driver);
 - LOG_DRIVER
- ll_entry_declare(struct part_driver, _name, part_driver);
 - U_BOOT_PART_TYPE
- ll_entry_declare(struct pci_driver_entry, _name, pci_driver_entry);
 - U_BOOT_PCI_DEVICE
- ll_entry_declare(struct phy_driver, _name, phy_driver);
 - U_BOOT_PHY_DRIVER

- ll_entry_declare(struct spl_image_loader, _name, spl_image_loader);
 - SPL_LOAD_IMAGE
- ll_entry_declare(struct optee_service, _name, optee_service);
 - OPTEE_SERVICE_DRIVER
- ll_entry_declare(struct fuzz_test, _name, fuzz_tests);
 - FUZZ_TEST
- ll_entry_declare(struct unit_test, _name, ut##_suite);
 - UNIT_TEST
- ll_entry_declare(struct usb_driver_entry, _name, usb_driver_entry);
 - U_BOOT_USB_DEVICE
- ll_entry_declare(struct w1_driver_entry, _name, w1_driver_entry);
 - U_BOOT_W1_DEVICE

12.1.2 u_boot_list_1 类型

暂未使用

12.1.3 u_boot_list_3 类型

暂未使用

12.2 uclass_driver

- UCLASS_DRIVER(bootdev)
- UCLASS_DRIVER(bootmeth)
- UCLASS_DRIVER(bootstd)
- UCLASS_DRIVER(usb_hub)
- UCLASS_DRIVER(usb_mass_storage)
- UCLASS_DRIVER(partition)
- UCLASS_DRIVER(adc)
- UCLASS_DRIVER(ahci)
- UCLASS_DRIVER(axi_emul)
- UCLASS_DRIVER(axi)
- UCLASS_DRIVER(blk)
- UCLASS_DRIVER(blkmap)
- UCLASS_DRIVER(efi_media)
- UCLASS_DRIVER(host)

- UCLASS_DRIVER(ide)
- UCLASS_DRIVER(bootcount)
- UCLASS_DRIVER(button)
- UCLASS_DRIVER(cache)
- UCLASS_DRIVER(clk)
- UCLASS_DRIVER(root)
- UCLASS_DRIVER(simple_bus)
- UCLASS_DRIVER(syscon)
- UCLASS_DRIVER(nop)
- UCLASS_DRIVER(cpu)
- UCLASS_DRIVER(hash)
- UCLASS_DRIVER(mod_exp)
- UCLASS_DRIVER(demo)
- UCLASS_DRIVER(dma)
- UCLASS_DRIVER(extcon)
- UCLASS_DRIVER(firmware)
- UCLASS_DRIVER(scmi_agent)
- UCLASS_DRIVER(fpga)
- UCLASS_DRIVER(fuzzing_engine)
- UCLASS_DRIVER(fwu_mdata)
- UCLASS_DRIVER(gpio)
- UCLASS_DRIVER(hwspinlock)
- UCLASS_DRIVER(i2c_emul)
- UCLASS_DRIVER(i2c_emul_parent)
- UCLASS_DRIVER(i2c)
- UCLASS_DRIVER(i2c_generic)
- UCLASS_DRIVER(i2c_mux)
- UCLASS_DRIVER(keyboard)
- UCLASS_DRIVER(iommu)
- UCLASS_DRIVER(led)
- UCLASS_DRIVER(mailbox)
- UCLASS_DRIVER(memory)

- UCLASS_DRIVER(cros_ec)
- UCLASS_DRIVER(fs_loader)
- UCLASS_DRIVER(i2c_eeprom)
- UCLASS_DRIVER(irq)
- UCLASS_DRIVER(misc)
- UCLASS_DRIVER(p2sb)
- UCLASS_DRIVER(pwrseq)
- UCLASS_DRIVER(qfw)
- UCLASS_DRIVER(mmc)
- UCLASS_DRIVER(mtd)
- UCLASS_DRIVER(nvmxip)
- UCLASS_DRIVER(spi_flash)
- UCLASS_DRIVER(mux)
- UCLASS_DRIVER(eth_phy_generic)
- UCLASS_DRIVER(nvme)
- UCLASS_DRIVER(pch)
- UCLASS_DRIVER(pci_emul)
- UCLASS_DRIVER(pci_emul_parent)
- UCLASS_DRIVER(pci)
- UCLASS_DRIVER(pci_generic)
- UCLASS_DRIVER(pci_ep)
- UCLASS_DRIVER(phy)
- UCLASS_DRIVER(pinconfig)
- UCLASS_DRIVER(pinctrl)
- UCLASS_DRIVER(acpi_pmc)
- UCLASS_DRIVER(power_domain)
- UCLASS_DRIVER(pmic)
- UCLASS_DRIVER(regulator)
- UCLASS_DRIVER(pwm)
- UCLASS_DRIVER(ram)
- UCLASS_DRIVER(reboot_mode)
- UCLASS_DRIVER(rproc)

- UCLASS_DRIVER(reset)
- UCLASS_DRIVER(rng)
- UCLASS_DRIVER rtc)
- UCLASS_DRIVER(scsi)
- UCLASS_DRIVER(serial)
- UCLASS_DRIVER(smem)
- UCLASS_DRIVER(soc)
- UCLASS_DRIVER(audio_codec)
- UCLASS_DRIVER(i2s)
- UCLASS_DRIVER(sound)
- UCLASS_DRIVER(spi_emul)
- UCLASS_DRIVER(spi)
- UCLASS_DRIVER(spi_generic)
- UCLASS_DRIVER(spmi)
- UCLASS_DRIVER(sysinfo)
- UCLASS_DRIVER(sysreset)
- UCLASS_DRIVER(tee)
- UCLASS_DRIVER(thermal)
- UCLASS_DRIVER(timer)
- UCLASS_DRIVER(tpm)
- UCLASS_DRIVER(ufs)
- UCLASS_DRIVER(usb_emul)
- UCLASS_DRIVER(usb_gadget_generic)
- UCLASS_DRIVER(usb)
- UCLASS_DRIVER(usb_dev_generic)
- UCLASS_DRIVER(backlight)
- UCLASS_DRIVER(video_bridge)
- UCLASS_DRIVER(display)
- UCLASS_DRIVER(dsi_host)
- UCLASS_DRIVER(panel)
- UCLASS_DRIVER(vidconsole)
- UCLASS_DRIVER(video)

- UCLASS_DRIVER(video_osd)
- UCLASS_DRIVER(virtio)
- UCLASS_DRIVER(w1_eeprom)
- UCLASS_DRIVER(w1)
- UCLASS_DRIVER(wdt)
- UCLASS_DRIVER(pvblock)
- UCLASS_DRIVER(ecdsa)
- UCLASS_DRIVER(efi)
- UCLASS_DRIVER(dsa)
- UCLASS_DRIVER(ethernet)
- UCLASS_DRIVER(mdio_mux)
- UCLASS_DRIVER(mdio)
- UCLASS_DRIVER(fdt_dummy)

12.3 driver_info

- U_BOOT_DRVINFO(imx6_thermal)
- U_BOOT_DRVINFO(imx7_thermal)
- U_BOOT_DRVINFO(lpc32xx_gpios)
- U_BOOT_DRVINFO(am33xx_spi)
- U_BOOT_DRVINFO(stm32mp_rom_api_ecdsa)
- U_BOOT_DRVINFO(ns16550_com1)
- U_BOOT_DRVINFO(tegra_gpios)
- U_BOOT_DRVINFO(soc_xilinx_versal_net)
- U_BOOT_DRVINFO(soc_xilinx_versal)
- U_BOOT_DRVINFO(soc_xilinx_zynqmp)
- U_BOOT_DRVINFO(integrator_serials)
- U_BOOT_DRVINFO(total_compute_serials)
- U_BOOT_DRVINFO(vexpress_serials)
- U_BOOT_DRVINFO(at91sam9260_serial)
- U_BOOT_DRVINFO(thunderx_serial0)
- U_BOOT_DRVINFO(cm_fx6_serial)
- U_BOOT_DRVINFO(omapl138_uart)
- U_BOOT_DRVINFO(omapl138_mmc)

- U_BOOT_DRVINFO(ls1012a_pfe0)
- U_BOOT_DRVINFO(hikey_seriala)
- U_BOOT_DRVINFO(hikey960_serial0)
- U_BOOT_DRVINFO(poplar_serial)
- U_BOOT_DRVINFO(igep_uart)
- U_BOOT_DRVINFO(sniper_serial)
- U_BOOT_DRVINFO(at91sam9260_serial)
- U_BOOT_DRVINFO(stv09911_serials)
- U_BOOT_DRVINFO(coldfire_serial)
- U_BOOT_DRVINFO(am335x_eth)
- U_BOOT_DRVINFO(am335x_mmc0)
- U_BOOT_DRVINFO(am335x_mmc1)
- U_BOOT_DRVINFO(devkit8000_uart)
- U_BOOT_DRVINFO(mxc_serial)
- U_BOOT_DRVINFO(fsl_rsa)
- U_BOOT_DRVINFO(hash_sw)
- U_BOOT_DRVINFO(mod_exp_sw)
- U_BOOT_DRVINFO(button_kbd)
- U_BOOT_DRVINFO(qfw_sandbox)
- U_BOOT_DRVINFO(proc_3_demo)
- U_BOOT_DRVINFO(rtc_emul)
- U_BOOT_DRVINFO(serial_sandbox_non_fdt)
- U_BOOT_DRVINFO(smh_serial)
- U_BOOT_DRVINFO(sysreset_sandbox_non_fdt)
- U_BOOT_DRVINFO(sandbox_timer_non_fdt)
- U_BOOT_DRVINFO(sunxi_de2)
- U_BOOT_DRVINFO(sunxi_lcd)